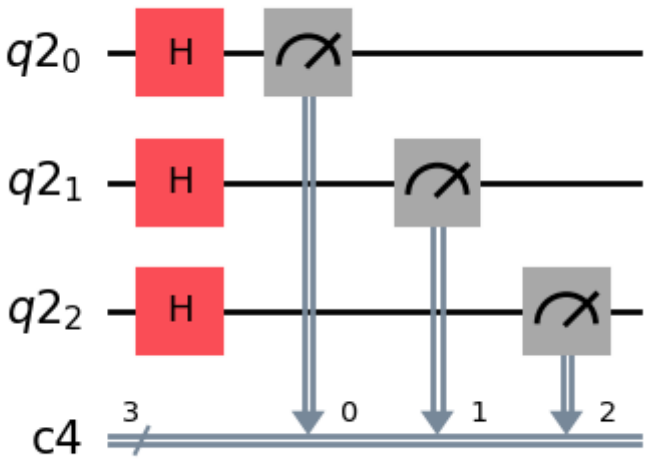
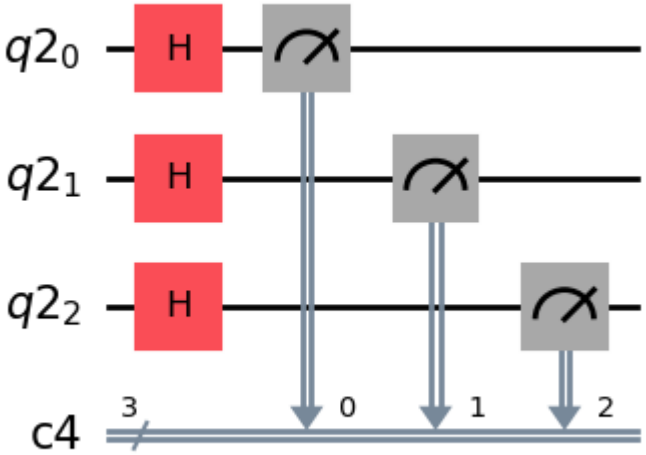
In [1]:
```
import sys
import numpy as np
from math import pi
import qiskit
from qiskit import ClassicalRegister, QuantumRegister, QuantumCircuit
from qiskit import transpile
from qiskit.circuit.library import RealAmplitudes
from qiskit.quantum_info import SparsePauliOp
from qiskit.visualization import plot_histogram,plot_bloch_multivector,
plot_histogram, array_to_latex
from qiskit.transpiler.preset_passmanagers import
generate_preset_pass_manager
from qiskit_ibm_runtime import QiskitRuntimeService
from qiskit.quantum_info import Statevector
from qiskit.circuit.library import MCXGate
from qiskit_aer import Aer, AerSimulator
from qiskit_quantuminspire.qi_provider import QIProvider
from qiskit_aer.primitives import SamplerV2 as Sampler
import cudaq
import qutip
import matplotlib.pylab as plt
from qiskit_aer.primitives import EstimatorV2
from qiskit_aer.primitives import SamplerV2
import warnings
warnings.filterwarnings('ignore')
```

Quantum Register and Classical Register

In [111…
```
# set up Quantum Register and Classical Register for 3 qubits
q = QuantumRegister(3)
c = ClassicalRegister(3)
# Create a Quantum Circuit
qc = QuantumCircuit(q, c)
qc.h(q)
qc.measure(q, c)

qc.draw('mpl')
```

Out[111…

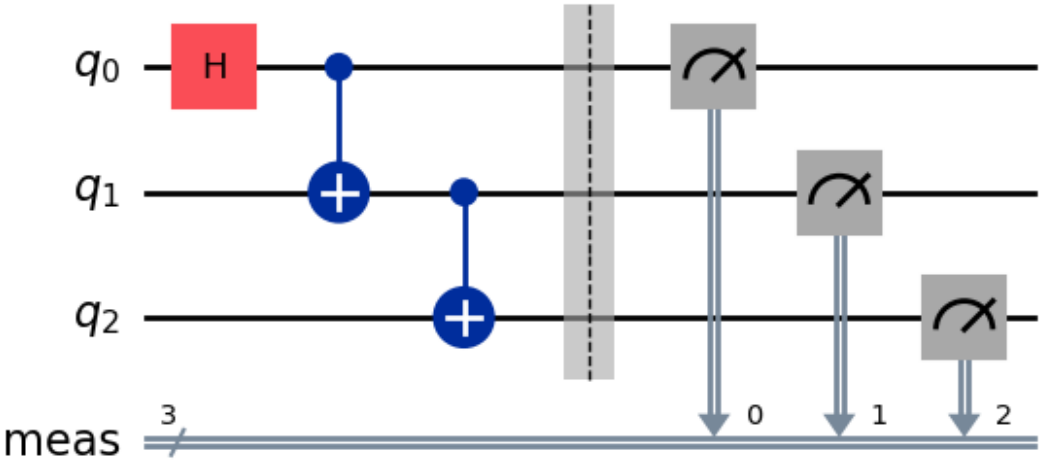


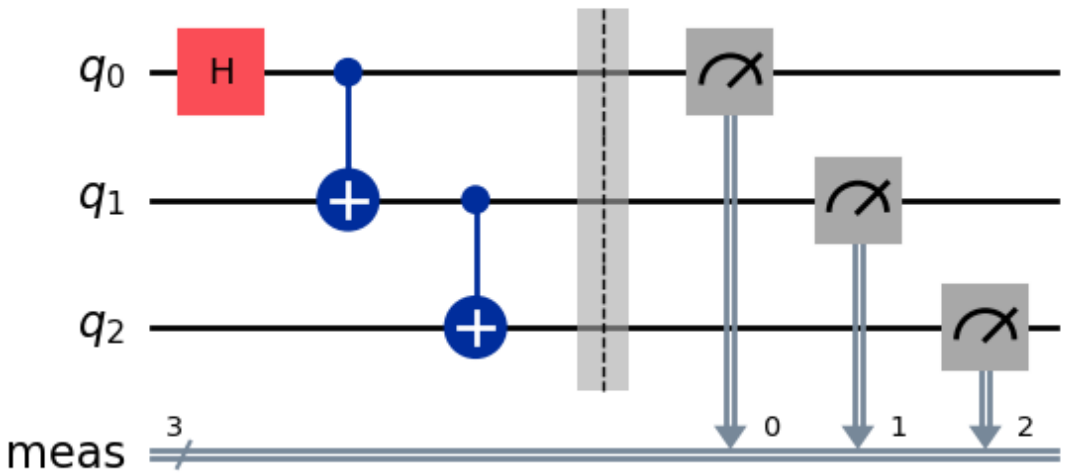


In [112…

```
circ = qiskit.QuantumCircuit(3)
circ.h(0)
circ.cx(0, 1)
circ.cx(1, 2)
circ.measure_all()
circ.draw('mpl')
```

Out[112…





In [113…

```
sampler = Sampler()
job = sampler.run([circ], shots=128)

# Perform an ideal simulation
result_ideal = job.result()
counts_ideal = result_ideal[0].data.meas.get_counts()
print('Counts(ideal):', counts_ideal)
```

```
Counts(ideal): {'111': 70, '000': 58}
```

In [114…

```
sim = AerSimulator()

psi1 = transpile(RealAmplitudes(num_qubits=2, reps=2), sim,
optimization_level=0)
psi2 = transpile(RealAmplitudes(num_qubits=2, reps=3), sim,
optimization_level=0)

H1 = SparsePauliOp.from_list([("II", 1), ("IZ", 2), ("XI", 3)])
H2 = SparsePauliOp.from_list([("IZ", 1)])
H3 = SparsePauliOp.from_list([("ZI", 1), ("ZZ", 1)])

theta1 = [0, 1, 1, 2, 3, 5]
theta2 = [0, 1, 1, 2, 3, 5, 8, 13]
theta3 = [1, 2, 3, 4, 5, 6]

estimator = EstimatorV2()

# calculate [ [<psi1(theta1)|H1|psi1(theta1)>,
#               <psi1(theta3)|H3|psi1(theta3)>],
#              [<psi2(theta2)|H2|psi2(theta2)>] ]
job = estimator.run(
    [
        (psi1, [H1, H3], [theta1, theta3]),
        (psi2, H2, theta2)
    ],
    precision=0.01
)
result = job.result()
print(f"expectation values : psi1 = {result[0].data.evs}, psi2 =
{result[1].data.evs}")
```

```
expectation values : psi1 = [ 1.5589867  -1.08519905], psi2 = 0.1709019582
2499799
```

In [115…

```python
# --------------------------
# Simulating using sampler
# --------------------------
from qiskit_aer.primitives import SamplerV2
from qiskit import QuantumCircuit

# create a Bell circuit
bell = QuantumCircuit(2)
bell.h(0)
bell.cx(0, 1)
bell.measure_all()

# create two parameterized circuits
pqc = RealAmplitudes(num_qubits=2, reps=2)
pqc.measure_all()
pqc = transpile(pqc, sim, optimization_level=0)
pqc2 = RealAmplitudes(num_qubits=2, reps=3)
pqc2.measure_all()
pqc2 = transpile(pqc2, sim, optimization_level=0)

theta1 = [0, 1, 1, 2, 3, 5]
theta2 = [0, 1, 2, 3, 4, 5, 6, 7]

# initialization of the sampler
sampler = SamplerV2()

# collect 128 shots from the Bell circuit
job = sampler.run([bell], shots=128)
job_result = job.result()
print(f"counts for Bell circuit :
{job_result[0].data.meas.get_counts()}")

# run a sampler job on the parameterized circuits
job2 = sampler.run([(pqc, theta1), (pqc2, theta2)])
job_result = job2.result()
print(f"counts for parameterized circuit :
{job_result[0].data.meas.get_counts()}")
```

```
counts for Bell circuit : {'00': 56, '11': 72}
counts for parameterized circuit : {'01': 365, '11': 437, '10': 96, '00':
126}
```

In [116…

```python
@cudaq.kernel
def kernel():
    A = cudaq.qubit()
    B = cudaq.qvector(3)
    C = cudaq.qvector(5)
```

In [117…

```python
N = 2
@cudaq.kernel
def kernel(N: int):
    register = cudaq.qvector(N)


# Passing complex vectors as parameters
c = [.707 + 0j, 0 - .707j]
```

```
@cudaq.kernel
def kernel(vec: list[complex]):
    q = cudaq.qvector(vec)
```

In [118…

```
c = np.array([0.70710678 + 0j, 0., 0., 0.70710678],
dtype=cudaq.complex())

@cudaq.kernel
def kernel():
    q = cudaq.qvector(c)
```

In [119…

```
# Define as CUDA-Q amplitudes
c = cudaq.amplitudes([0.70710678 + 0j, 0., 0., 0.70710678])
@cudaq.kernel
def kernel():
    q = cudaq.qvector(c)
```

In [ ]:

In [120…

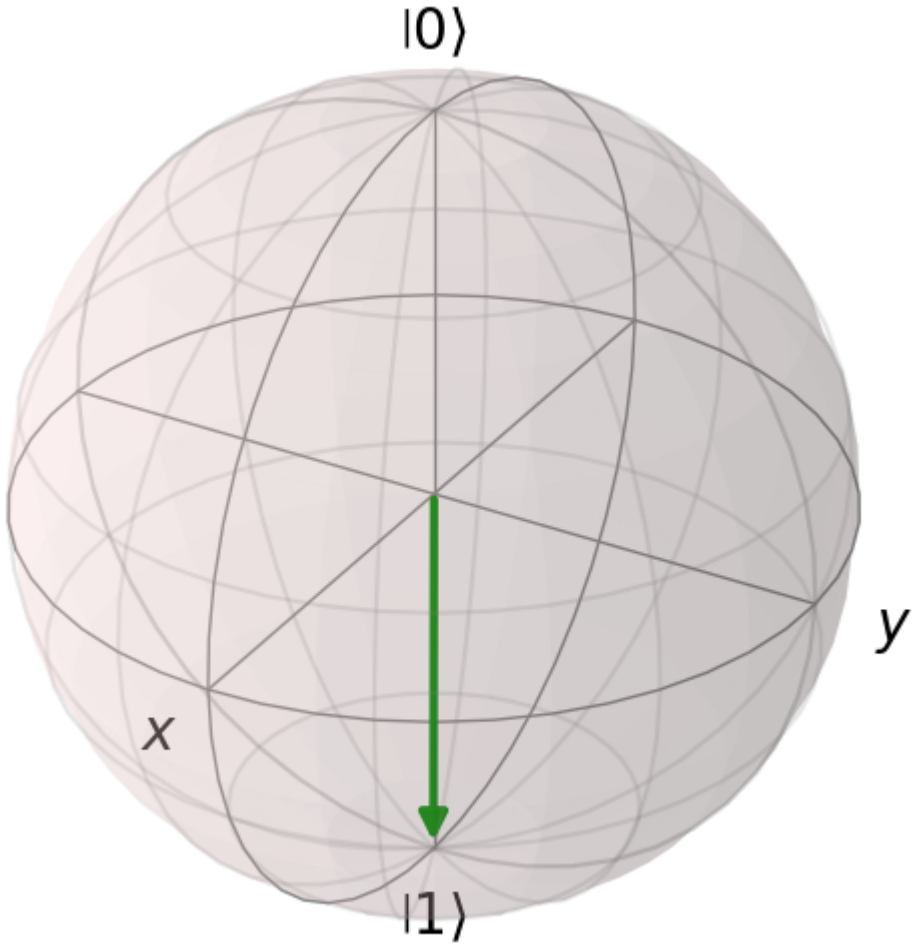
```
## Retry the subsequent cells by setting the target to density matrix
simulator.
cudaq.set_target("density-matrix-cpu")
%matplotlib inline

@cudaq.kernel
def kernel(angles: np.ndarray):
    qubit = cudaq.qubit()
    rz(angles[0], qubit)
    rx(angles[1], qubit)
    rz(angles[2], qubit)
```

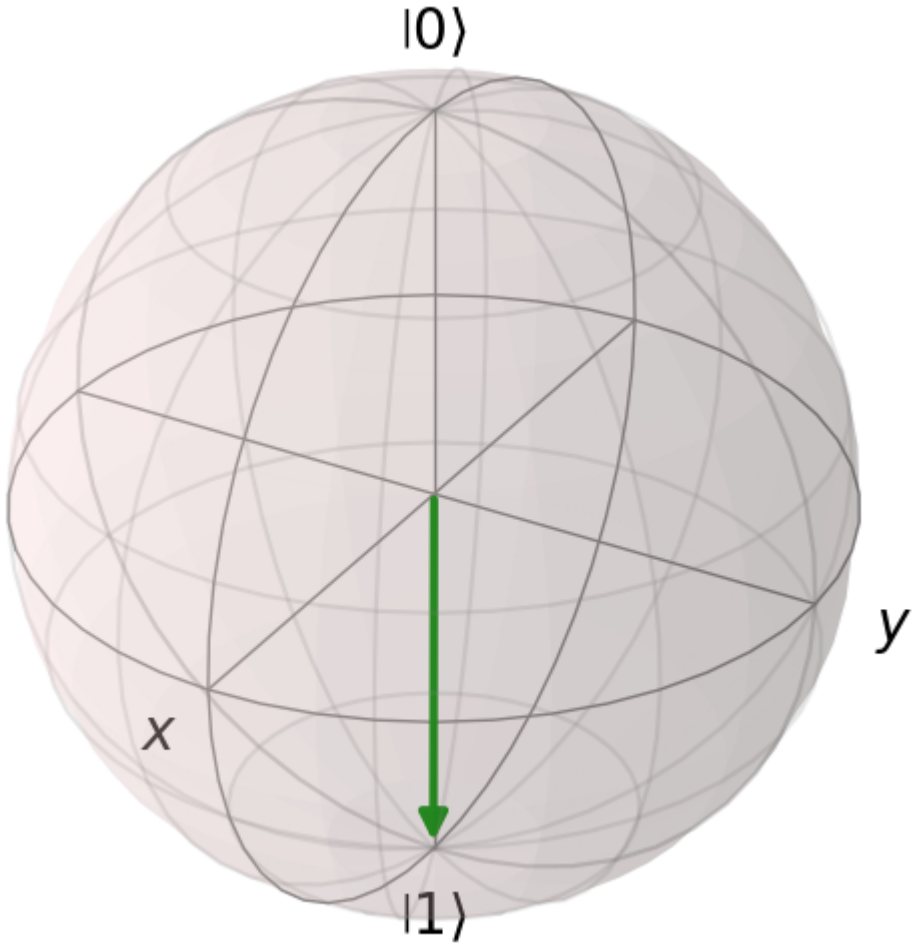
In [121…

```
rng = np.random.default_rng(seed=11)
blochSphereList = []
for _ in range(4):
    angleList = rng.random(3) * 2 * np.pi
    sph = cudaq.add_to_bloch_sphere(cudaq.get_state(kernel, angleList))
    blochSphereList.append(sph)
```

In [122…

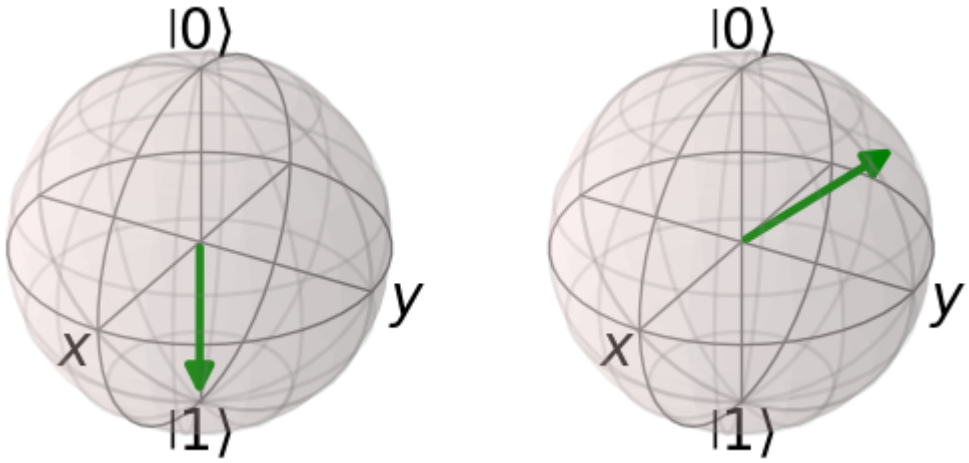
```
cudaq.show(blochSphereList[0])
```


|0⟩
y
x
|1⟩



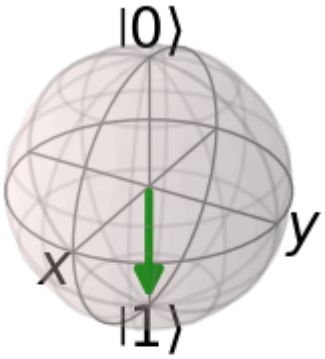
In [123…

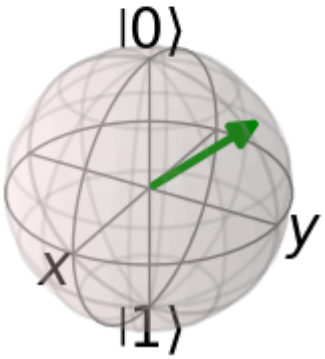
```
cudaq.show(blochSphereList[:2], nrows=1, ncols=2)
```



In [124…

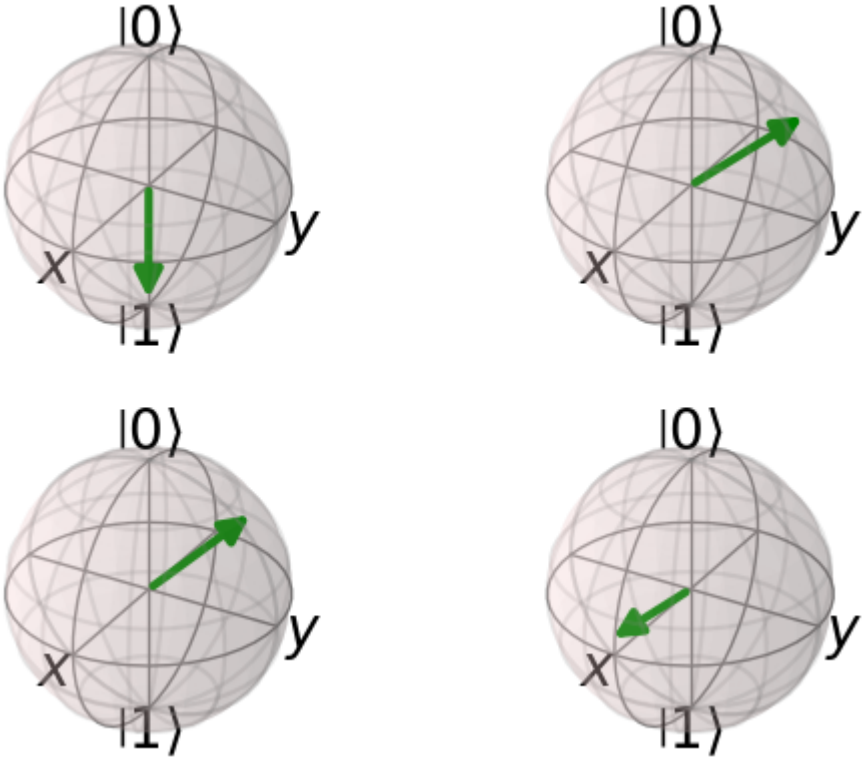
```
cudaq.show(blochSphereList[:2], nrows=2, ncols=1)
```





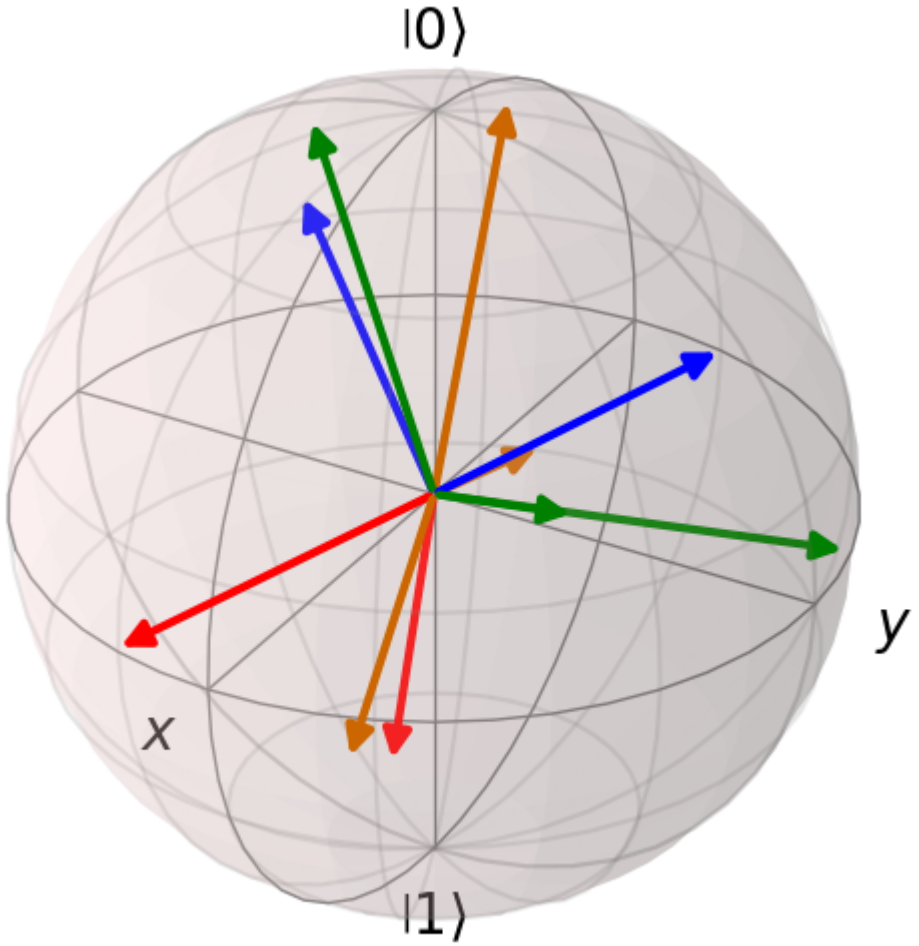
In [125…

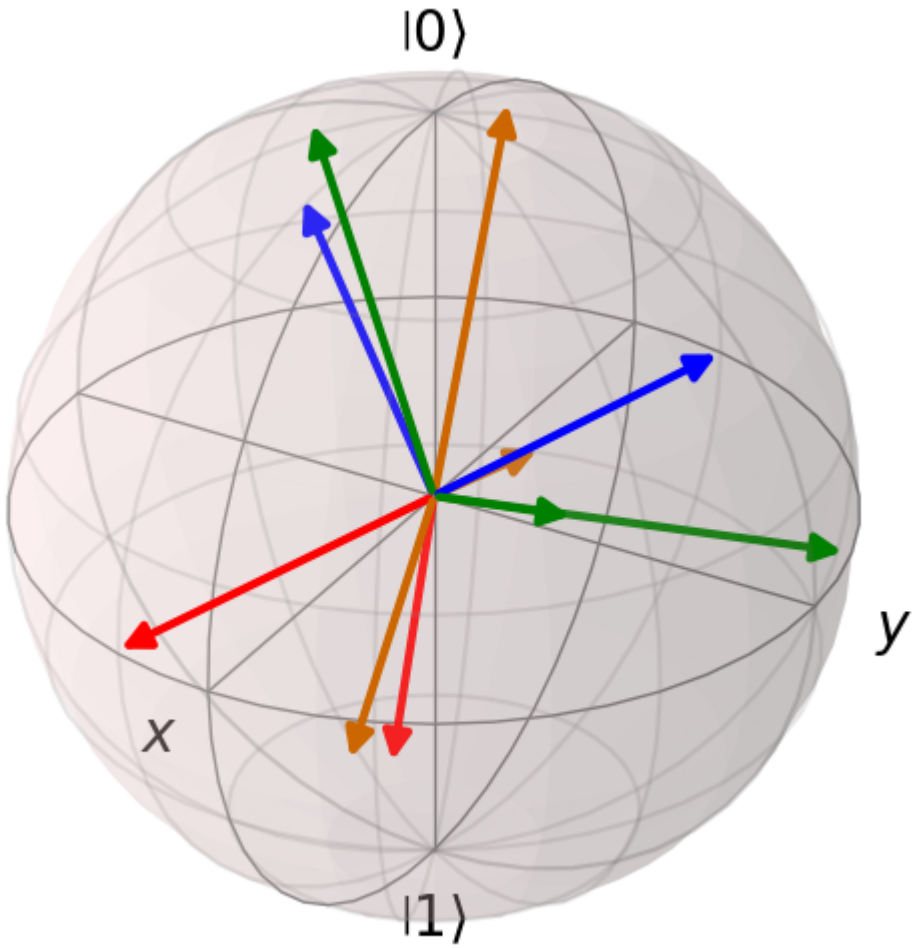
```
cudaq.show(blochSphereList[:], nrows=2, ncols=2)
```



In [126…

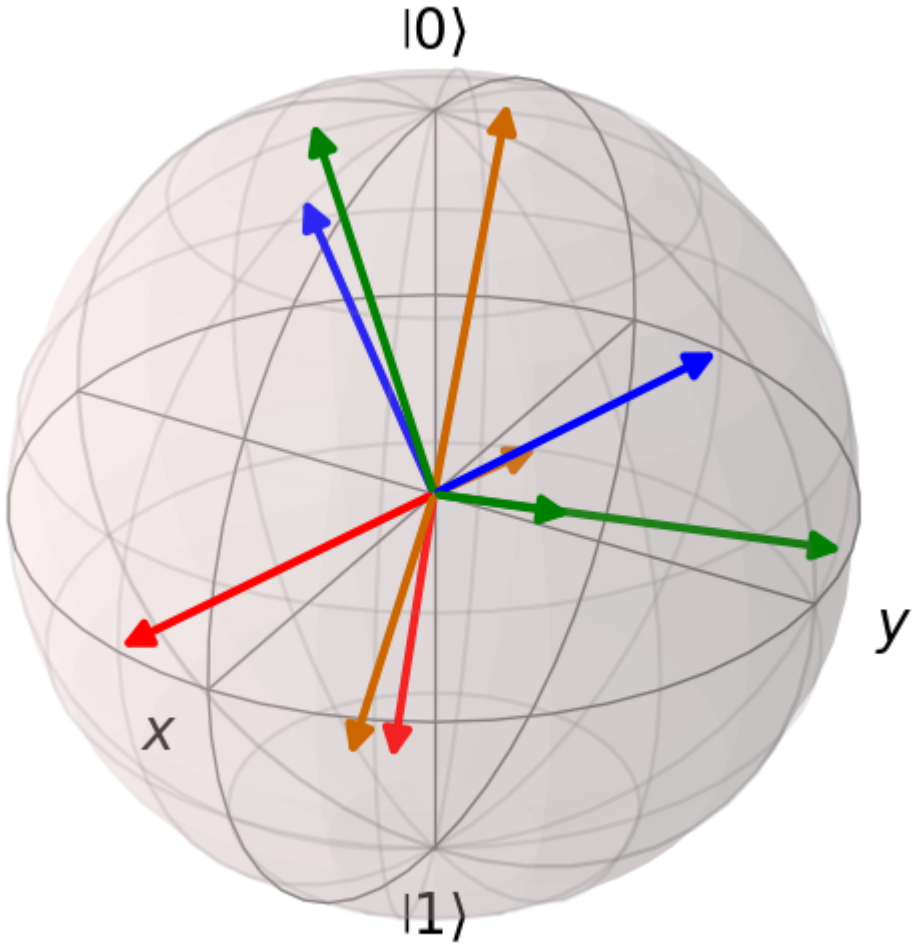
```
rng = np.random.default_rng(seed=47)
blochSphere = qutip.Bloch()
for _ in range(10):
    angleList = rng.random(3) * 2 * np.pi
    sph = cudaq.add_to_bloch_sphere(cudaq.get_state(kernel, angleList),
blochSphere)
blochSphere.show()
```

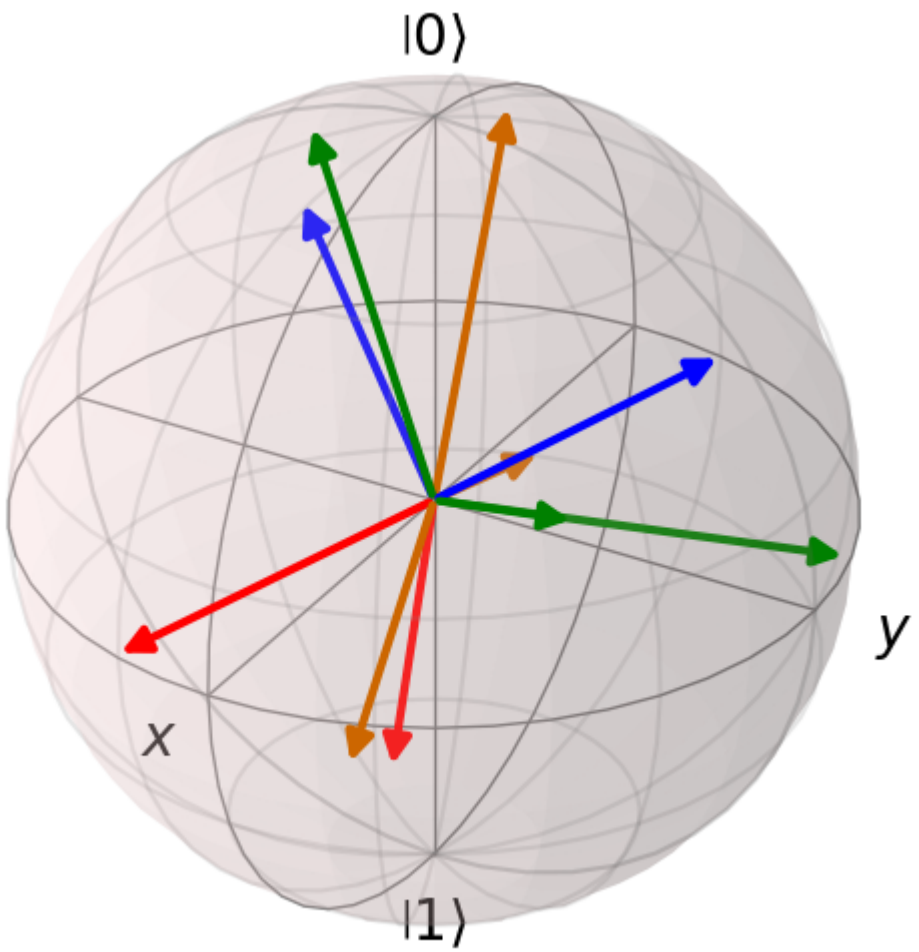

|0⟩
y
x
|1⟩



In [127…

```
blochSphere.show()
```


|0⟩
y
x
|1⟩



In [128…

```
@cudaq.kernel
def kernel_to_draw():
    q = cudaq.qvector(4)
    h(q)
    x.ctrl(q[0], q[1])
    y.ctrl([q[0], q[1]], q[2])
    z(q[2])

    swap(q[0], q[1])
    swap(q[0], q[3])
    swap(q[1], q[2])

    r1(3.14159, q[0])
    tdg(q[1])
    s(q[2])
```

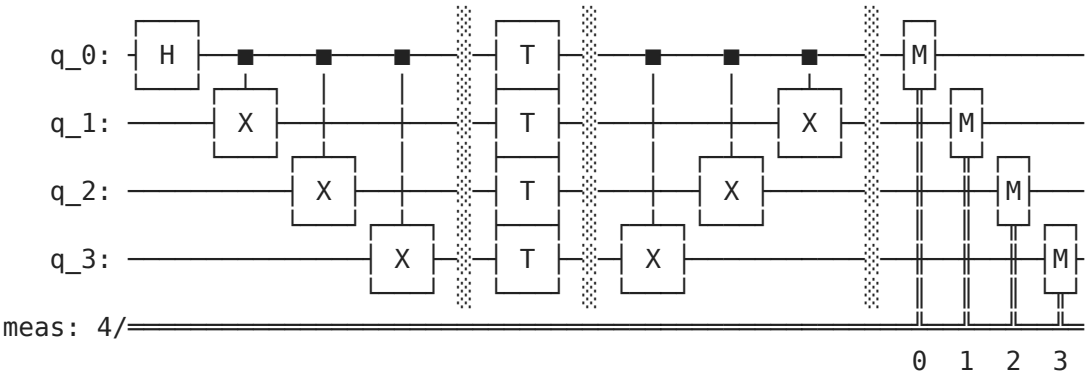
In [129…

```
#print(cudaq.draw('latex', kernel_to_draw))
```

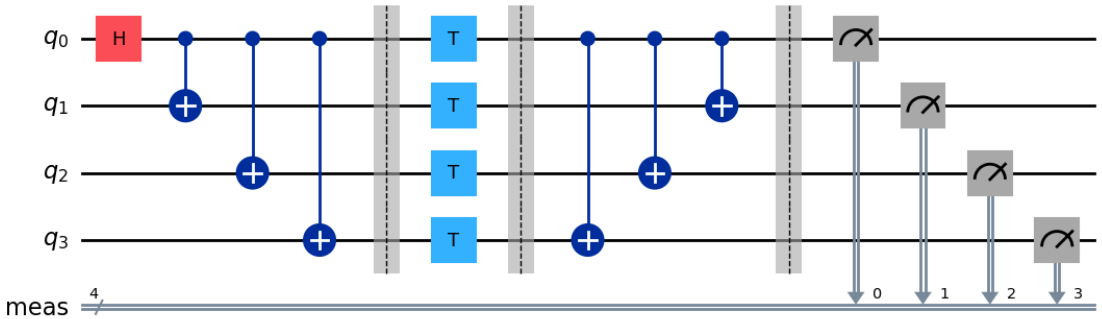
In [130…

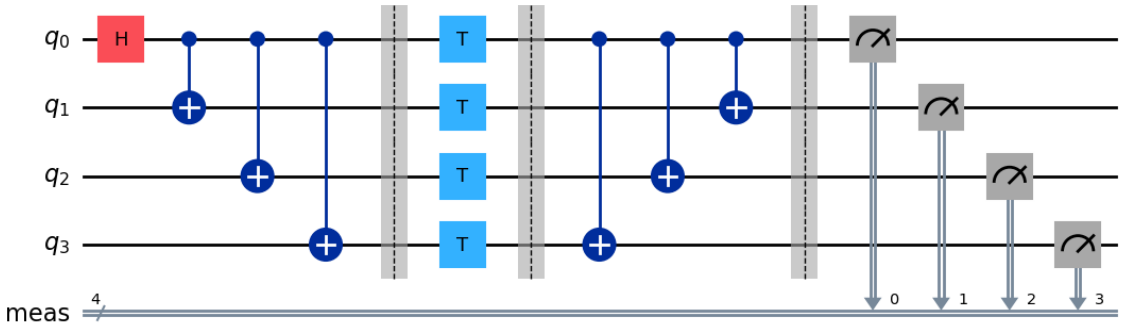
```
qc = QuantumCircuit(4)
qc.h(0)
qc.cx(0, 1)
qc.cx(0, 2)
qc.cx(0, 3)

qc.barrier()

qc.t(0)
qc.t(1)
```

```
qc.t(2)
qc.t(3)

qc.barrier()

qc.cx(0, 3)
qc.cx(0, 2)
qc.cx(0, 1)

qc.measure_all()
print(qc)
qc.draw()
```



Out[130…

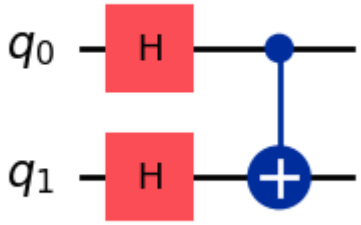


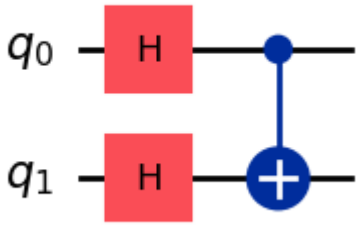


In [131…

```
qc = QuantumCircuit(2)
qc.h(0)
qc.h(1)
qc.cx(0,1)
qc.draw()
```

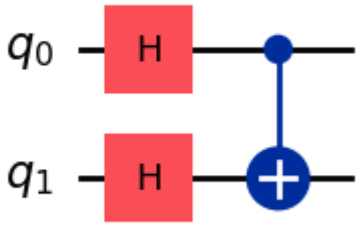
Out[131…





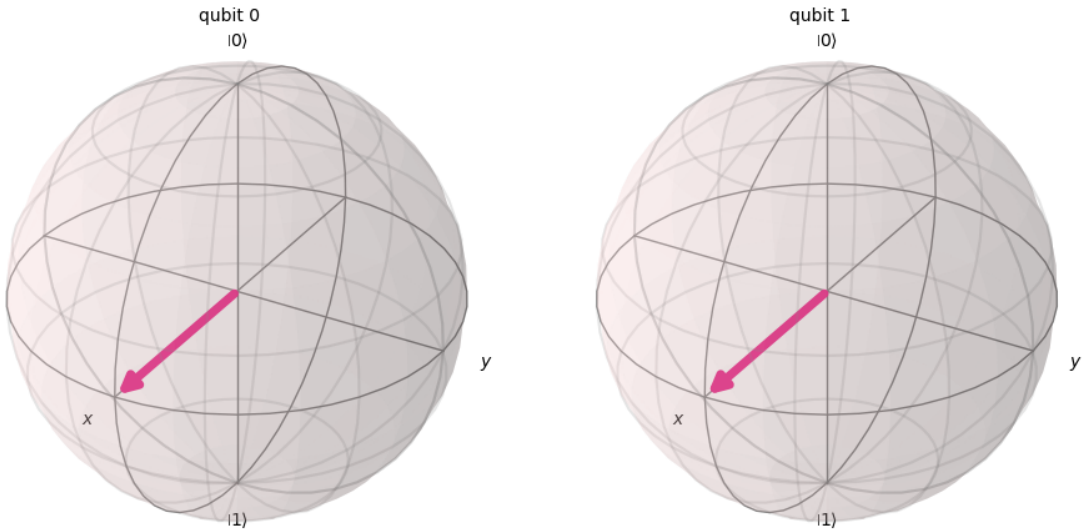
In [132…

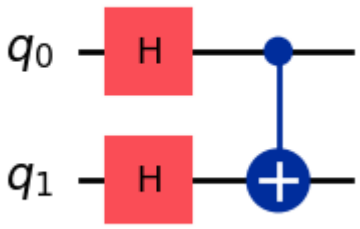
```
qc = QuantumCircuit(2)
qc.h(0)
qc.h(1)
qc.cx(0,1)
display(qc.draw())  # `display` is a command for Jupyter notebooks
                    # similar to `print`, but for rich content

# Let's see the result
svsim = Aer.get_backend('aer_simulator')
qc.save_statevector()
qobj = transpile(qc, svsim)
final_state = svsim.run(qobj).result().get_statevector()
display(array_to_latex(final_state, prefix="\\text{Statevector} = "))
plot_bloch_multivector(final_state)
```

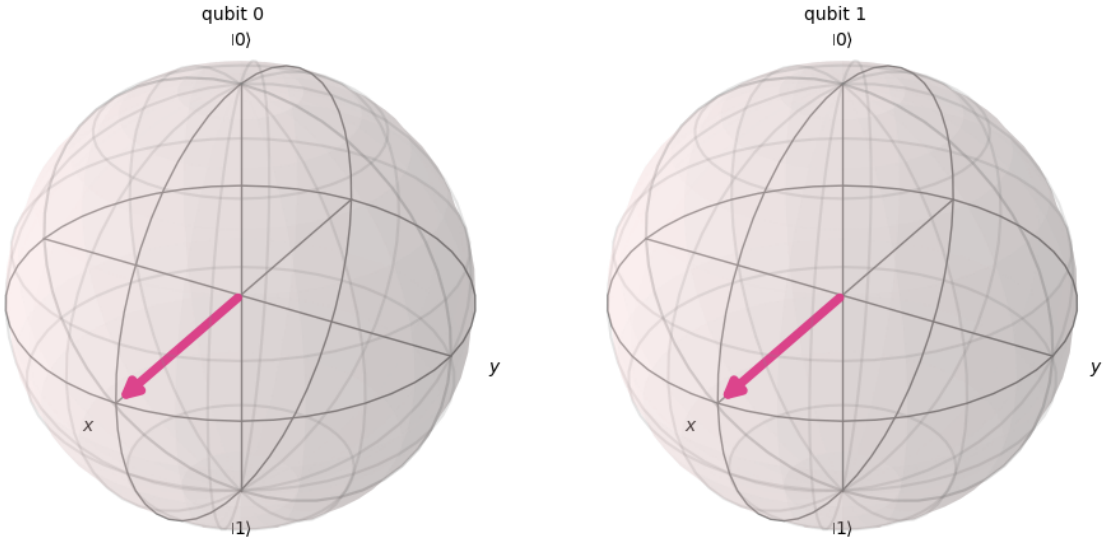


$$\text{Statevector} = \begin{bmatrix} \frac{1}{2} & \frac{1}{2} & \frac{1}{2} & \frac{1}{2} \end{bmatrix}$$

Out[132…







In [133…

```
bell = QuantumCircuit(2)
bell.h(0)
bell.cx(0, 1)
bell.measure_all()

sampler = Sampler()
sampler.options.default_shots = 1024
job = sampler.run([bell])
print(f"Job ID is {job.job_id()}")
pub_result = job.result()[0]
print(f"Counts for the meas output register:
{pub_result.data.meas.get_counts()}")
```

```
Job ID is 49dacfdf-7ac9-49ee-b972-fa80fd5daca7
Counts for the meas output register: {'00': 496, '11': 528}
```

In [134…

```python
sim = AerSimulator()


psi1 = transpile(RealAmplitudes(num_qubits=2, reps=2), sim,
optimization_level=0)
psi2 = transpile(RealAmplitudes(num_qubits=2, reps=3), sim,
optimization_level=0)

H1 = SparsePauliOp.from_list([("II", 1), ("IZ", 2), ("XI", 3)])
H2 = SparsePauliOp.from_list([("IZ", 1)])
H3 = SparsePauliOp.from_list([("ZI", 1), ("ZZ", 1)])

theta1 = [0, 1, 1, 2, 3, 5]
theta2 = [0, 1, 1, 2, 3, 5, 8, 13]
theta3 = [1, 2, 3, 4, 5, 6]

estimator = EstimatorV2()

# calculate [ [<psi1(theta1)|H1|psi1(theta1)>,
#               <psi1(theta3)|H3|psi1(theta3)>],
#              [<psi2(theta2)|H2|psi2(theta2)>] ]
job = estimator.run(
    [
        (psi1, [H1, H3], [theta1, theta3]),
        (psi2, H2, theta2)
    ],
    precision=0.01
)
result = job.result()
print(f"expectation values : psi1 = {result[0].data.evs}, psi2 =
{result[1].data.evs}")

# create a Bell circuit
bell = QuantumCircuit(2)
bell.h(0)
bell.cx(0, 1)
bell.measure_all()

# create two parameterized circuits
pqc = RealAmplitudes(num_qubits=2, reps=2)
pqc.measure_all()
pqc = transpile(pqc, sim, optimization_level=0)
pqc2 = RealAmplitudes(num_qubits=2, reps=3)
pqc2.measure_all()
pqc2 = transpile(pqc2, sim, optimization_level=0)

theta1 = [0, 1, 1, 2, 3, 5]
theta2 = [0, 1, 2, 3, 4, 5, 6, 7]

# initialization of the sampler
sampler = SamplerV2()

# collect 128 shots from the Bell circuit
job = sampler.run([bell], shots=128)
job_result = job.result()
print(f"counts for Bell circuit :
{job_result[0].data.meas.get_counts()}")
```

```
# run a sampler job on the parameterized circuits
job2 = sampler.run([(pqc, theta1), (pqc2, theta2)])
job_result = job2.result()
print(f"counts for parameterized circuit :
{job_result[0].data.meas.get_counts()}")
```

```
expectation values : psi1 = [ 1.56921224 -1.09004461], psi2 = 0.1750868697
1005502
counts for Bell circuit : {'11': 64, '00': 64}
counts for parameterized circuit : {'01': 377, '11': 438, '10': 101, '00':
108}
```

In [135…

```
# 1. Create a Quantum Circuit
qc = QuantumCircuit(2)
qc.h(0)
qc.cx(0, 1)
qc.measure_all()
```

In [136…

```
# 2. Initialize the AerSimulator
simulator = AerSimulator()

# 3. Transpile the circuit for the simulator
compiled_circuit = transpile(qc, simulator)
```

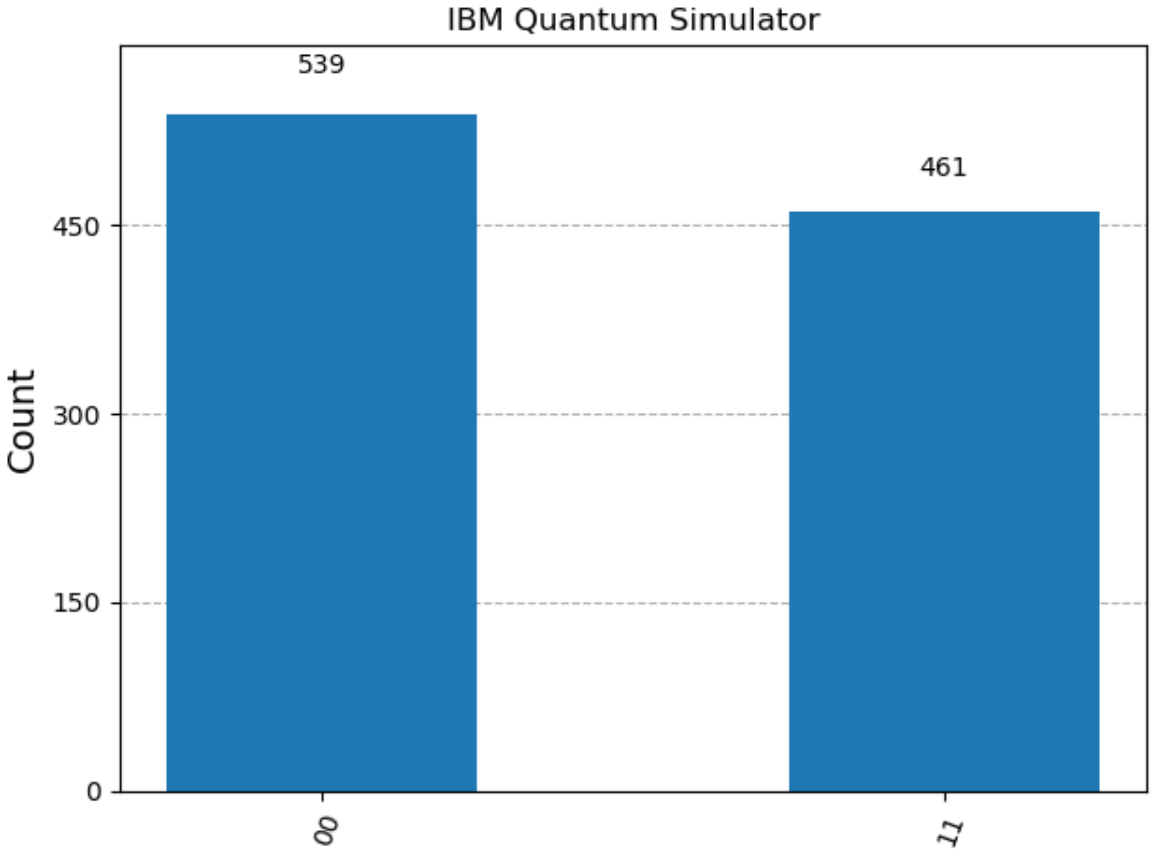
In [137…

```
# 4. Run the circuit and get results
job = simulator.run(compiled_circuit, shots=1000)
result = job.result()
counts = result.get_counts()

print("\nCounts:", counts)
```

```
Counts: {'11': 461, '00': 539}
```

In [138…

```
histogram = plot_histogram(counts,title="IBM Quantum Simulator")
```

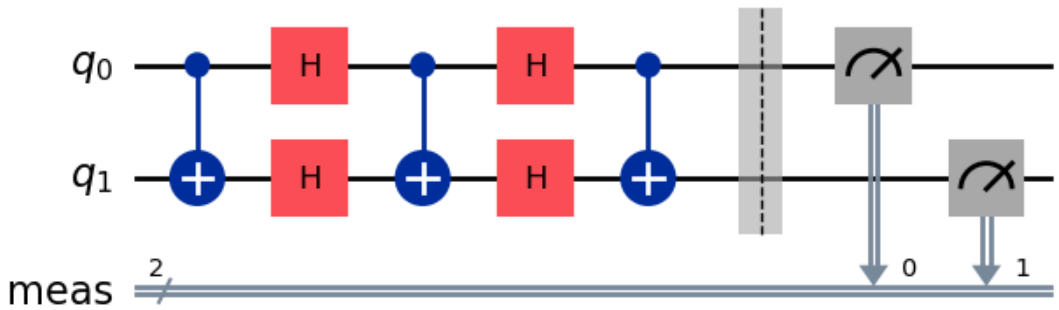


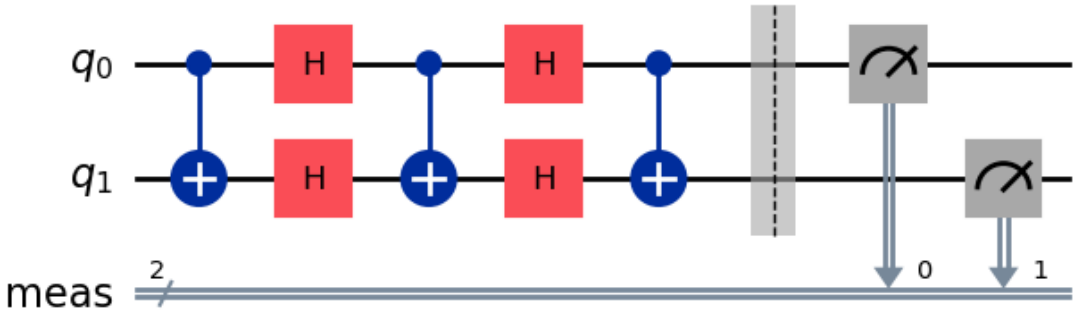
In [ ]:

In [139…

```
circuit = QuantumCircuit(2)
circuit.cx(0, 1)
circuit.h(0)
circuit.h(1)
circuit.cx(0, 1)
circuit.h(0)
circuit.h(1)
circuit.cx(0, 1)
circuit.measure_all()
circuit.draw()
```

Out[139…





In [140…

```
initialization_circuit = QuantumCircuit(2)
initialization_circuit.id(0)
initialization_circuit.x(1)
initialization_circuit.barrier()
```

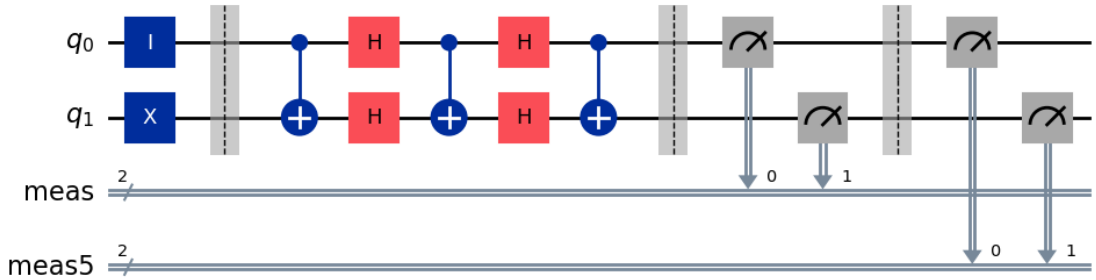
Out[140…

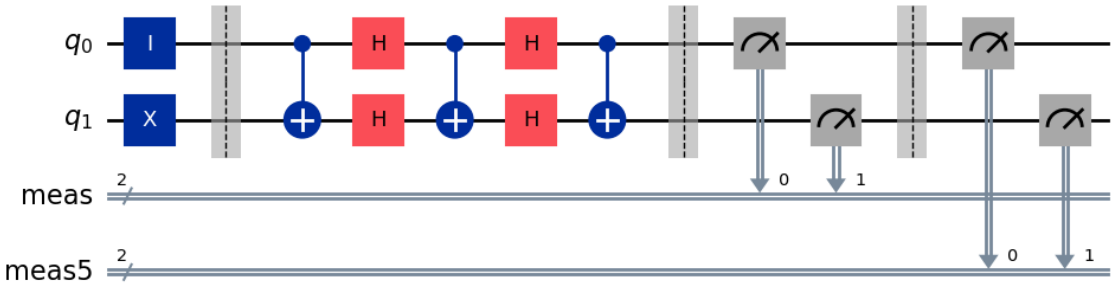
```
CircuitInstruction(operation=Instruction(name='barrier', num_qubits=2, n
um_clbits=0, params=[]), qubits=(<Qubit register=(2, "q"), index=0>, <Qu
bit register=(2, "q"), index=1>), clbits=())
```

In [141…

```
new_circuit = initialization_circuit.compose(circuit)
new_circuit.measure_all()
new_circuit.draw()
```

Out[141…

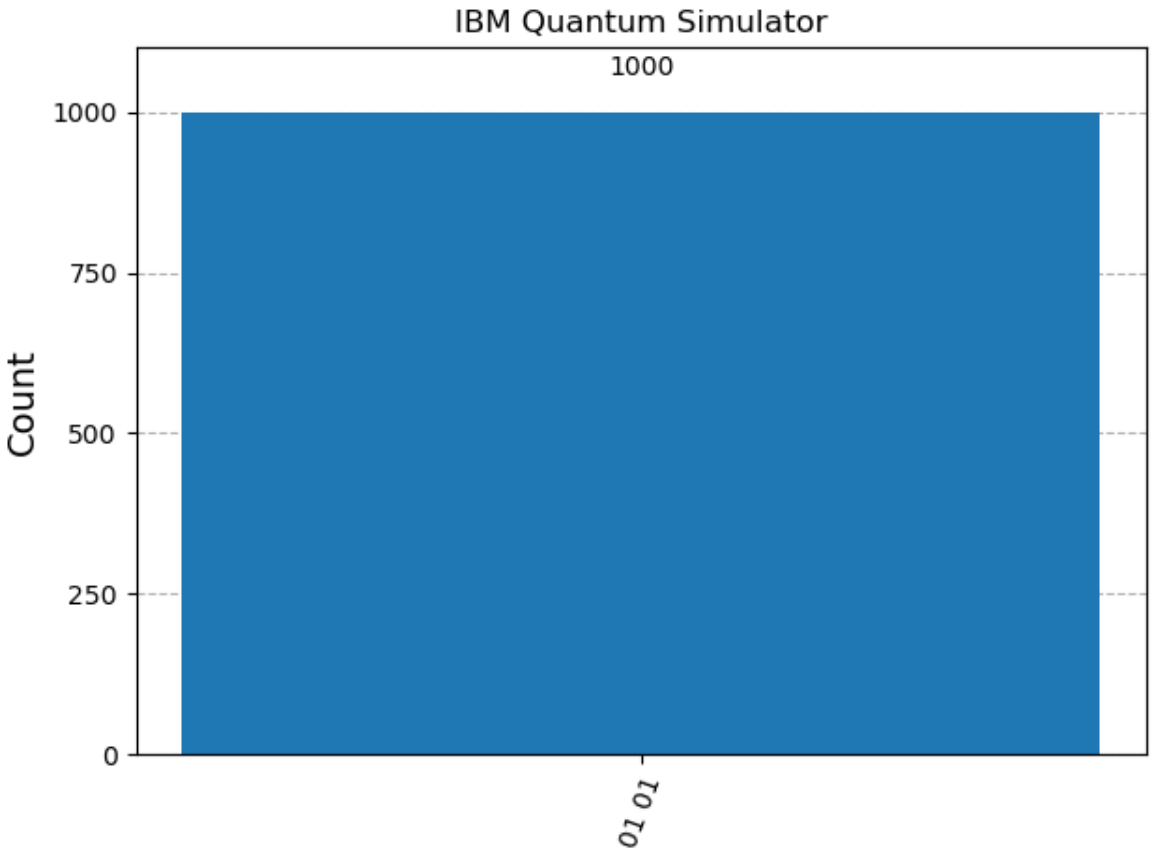




In [142…

```
simulator = AerSimulator()
compiled_new_circuit = transpile(new_circuit, simulator)
newjob = simulator.run(compiled_new_circuit, shots=1000)
nresult = newjob.result()
ncounts = nresult.get_counts()

print("\nCounts:", ncounts)

histogram = plot_histogram(ncounts,title="IBM Quantum Simulator")
```

```
Counts: {'01 01': 1000}
```



In [143…

```
provider = QIProvider()
print(provider.backends())
backend = provider.get_backend("QX emulator")
```

```
[<qiskit_quantuminspire.qi_backend.QIBackend object at 0x72773621ee40 (nam
e=QX emulator, id=1)>, <qiskit_quantuminspire.qi_backend.QIBackend object
at 0x7276acb28190 (name=Tuna-5, id=4)>, <qiskit_quantuminspire.qi_backend.
QIBackend object at 0x7276acb282d0 (name=Ry emulator, id=5)>, <qiskit_quan
tuminspire.qi_backend.QIBackend object at 0x7276cf0ede00 (name=Tuna-9, id=
6)>]
```

In [144…

```
circuit = QuantumCircuit(3, 10)
circuit.h(0)
circuit.cx(0, 1)
circuit.measure(0, 0)
circuit.measure(1, 1)
```

Out[144…

```
<qiskit.circuit.instructionset.InstructionSet at 0x7276d46b1120>
```

In [145…

```
backend = provider.get_backend(name="QX emulator")
job = backend.run(circuit, shots=1024)
result = job.result()
print(result)
```

```
Result(backend_name='QX emulator', backend_version='1.0.0', job_id='375518
', success=True, results=[ExperimentResult(shots=1024, success=True, meas_
level=2, data=ExperimentResultData(counts={'0x0': 502, '0x3': 522}), heade
r={'name': 'circuit-283', 'memory_slots': 10}, status=Experiment successfu
l)], date=None, status=Result successful, header=None, qobj_id='', system_
messages={})
```

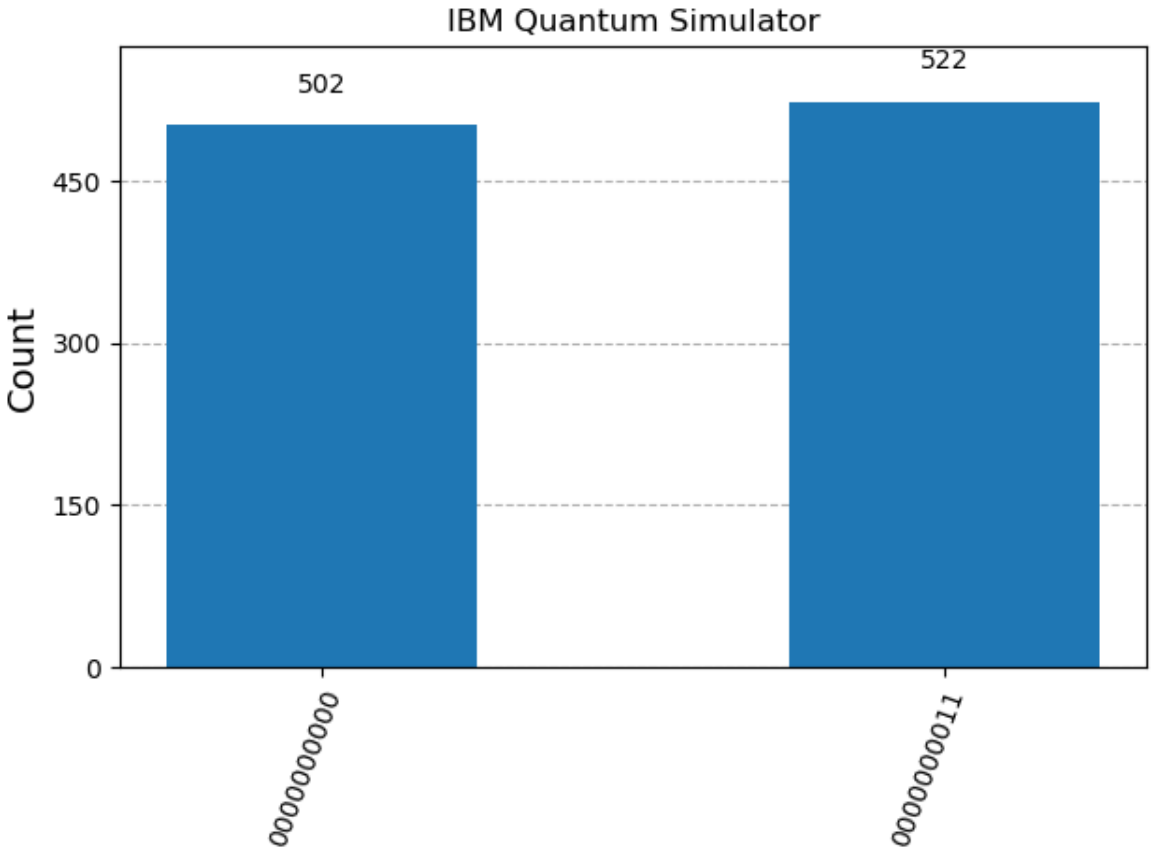
In [146…

```
counts = result.get_counts()
print("\nCounts:", counts)
```

```
Counts: {'0000000000': 502, '0000000011': 522}
```

In [147…

```
histogram = plot_histogram(counts,title="IBM Quantum Simulator")
```



In [148…

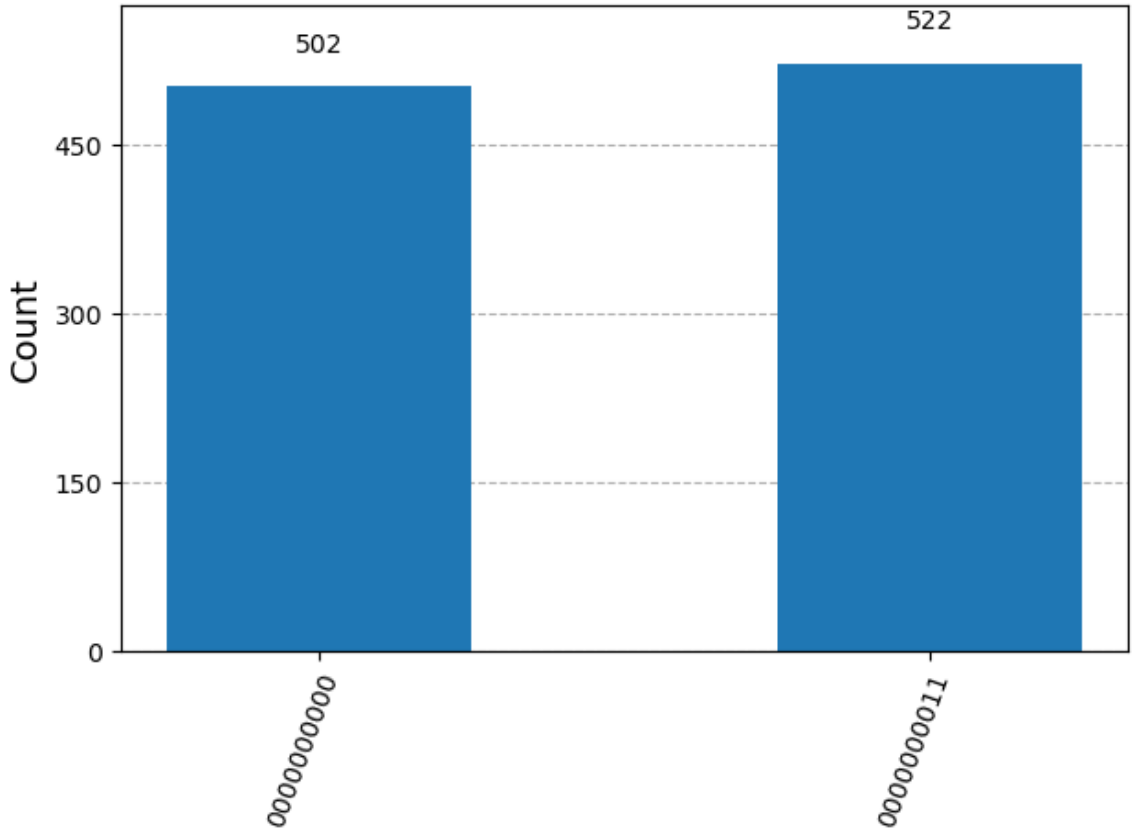
```
x = QuantumRegister(1, 'x')
y = QuantumRegister(1, 'y')
z = QuantumRegister(1, 'z')
v = QuantumRegister(1, 'v')
aux = QuantumRegister(2,'aux')
r = QuantumRegister(1, 'result')
qc = QuantumCircuit(x,y,z,v,aux,r)

qc.x(x)
qc.x(y)
qc.x(z)
qc.x(v)
qc.barrier()
qc.ccx(x,y,aux[0])
qc.ccx(aux[0],z,aux[1])
qc.ccx(aux[1],z,r)
qc.barrier()
qc.ccx(x,y,aux[0])
qc.ccx(aux[0],z,aux[1])
qc.ccx(aux[1],z,r)
#un-computing of the aux registers
qc.ccx(aux[0],z,aux[1])
qc.ccx(x,y,aux[0])

qc.measure_all()
qc.draw()

# Let's see the result
svsim = Aer.get_backend('aer_simulator')
```

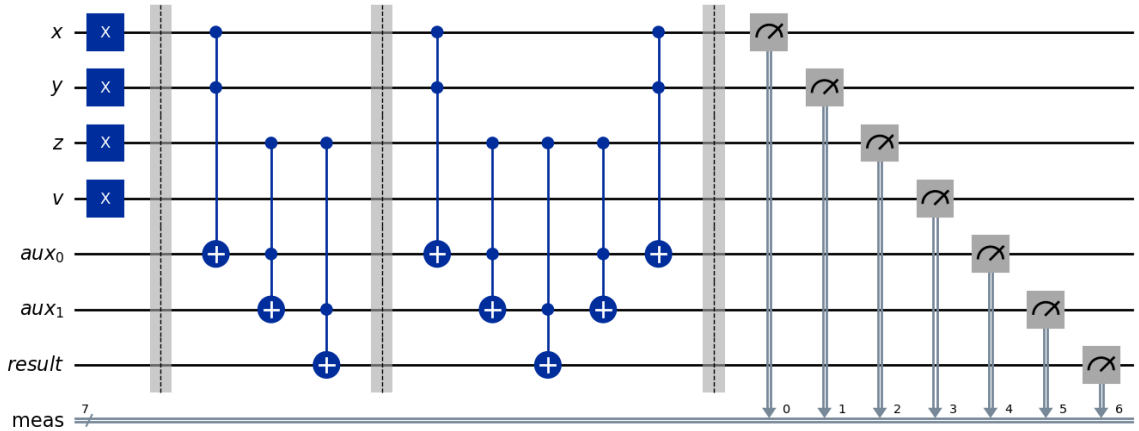
```
qc.save_statevector()
qobj = transpile(qc, svsim)
final_state = svsim.run(qobj).result().get_statevector()
display(array_to_latex(final_state, prefix="\\text{Statevector} = "))
plot_bloch_multivector(final_state)
counts = result.get_counts()


print("\nTotal count are:",counts)
plot_histogram(counts)
```

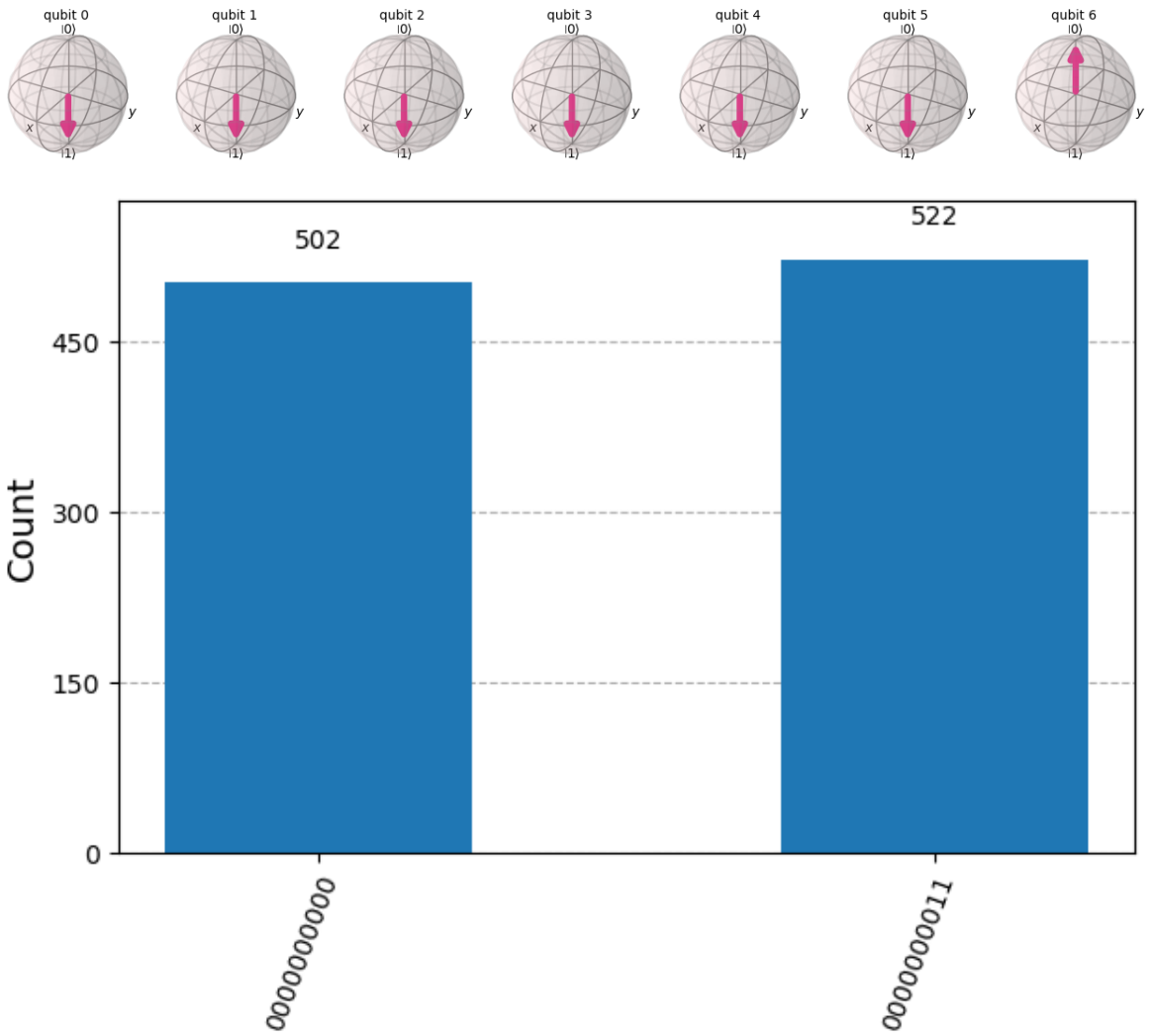
$$\text{Statevector} = \begin{bmatrix} 0 & 0 & 0 & 0 & \cdots & 0 & 0 & 0 \end{bmatrix}$$

```
Total count are: {'0000000000': 502, '0000000011': 522}
```

Out[148…







Simulating Qiskit circuit with Aer

In [149…

```
sim = AerSimulator()
psi1 = transpile(RealAmplitudes(num_qubits=2, reps=2), sim,
optimization_level=0)
psi2 = transpile(RealAmplitudes(num_qubits=2, reps=3), sim,
optimization_level=0)

H1 = SparsePauliOp.from_list([("II", 1), ("IZ", 2), ("XI", 3)])
H2 = SparsePauliOp.from_list([("IZ", 1)])
H3 = SparsePauliOp.from_list([("ZI", 1), ("ZZ", 1)])

theta1 = [0, 1, 1, 2, 3, 5]
theta2 = [0, 1, 1, 2, 3, 5, 8, 13]
theta3 = [1, 2, 3, 4, 5, 6]

estimator = EstimatorV2()

# calculate [ [<psi1(theta1)|H1|psi1(theta1)>,
#               <psi1(theta3)|H3|psi1(theta3)>],
#              [<psi2(theta2)|H2|psi2(theta2)>] ]
job = estimator.run(
    [
        (psi1, [H1, H3], [theta1, theta3]),
        (psi2, H2, theta2)
    ],
```

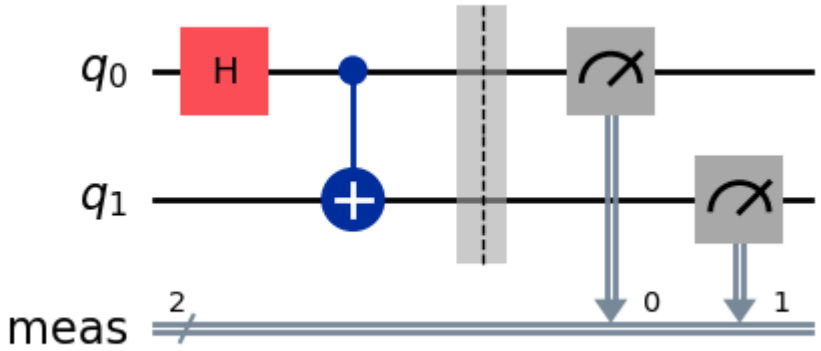
```
    precision=0.01
)
result = job.result()
print(f"expectation values : psi1 = {result[0].data.evs}, psi2 =
{result[1].data.evs}")
```

```
expectation values : psi1 = [ 1.54849249 -1.08765917], psi2 = 0.1776018031
0111345
```

In [150…

```
# create a Bell circuit
bell = QuantumCircuit(2)
bell.h(0)
bell.cx(0, 1)
bell.measure_all()
bell.draw()
# create two parameterized circuits
pqc = RealAmplitudes(num_qubits=2, reps=2)
pqc.measure_all()
pqc = transpile(pqc, sim, optimization_level=0)
pqc2 = RealAmplitudes(num_qubits=2, reps=3)
pqc2.measure_all()
pqc2 = transpile(pqc2, sim, optimization_level=0)

theta1 = [0, 1, 1, 2, 3, 5]
theta2 = [0, 1, 2, 3, 4, 5, 6, 7]

# initialization of the sampler
sampler = SamplerV2()

# collect 128 shots from the Bell circuit
job = sampler.run([bell], shots=128)
job_result = job.result()
print(f"counts for Bell circuit :
{job_result[0].data.meas.get_counts()}")

# run a sampler job on the parameterized circuits
job2 = sampler.run([(pqc, theta1), (pqc2, theta2)])
job_result = job2.result()
print(f"counts for parameterized circuit :
{job_result[0].data.meas.get_counts()}")
```

```
counts for Bell circuit : {'00': 70, '11': 58}
counts for parameterized circuit : {'11': 404, '00': 137, '01': 392, '10':
91}
```



Quantum oracle

In [151…

```
qc = QuantumCircuit(3)
#Difusor
qc.h([0,1,2])
qc.x([0,1,2])
qc.h(0)
qc.ccx(1,2,0)
qc.h(0)
qc.x([0,1,2])
qc.h([0,1,2])

#Multiply with (-1)
qc.rz(2*pi,0)
qc.rz(2*pi,1)
qc.rz(2*pi,2)

simulator = AerSimulator()
qc.save_unitary()
qobj = transpile(qc, simulator)
unitary = simulator.run(qobj).result().get_unitary()
print("\nSize of the unitary matrix:",np.asarray(unitary).shape)
array_to_latex(unitary, prefix="\\text{G3 = }\n")
```

```
Size of the unitary matrix: (8, 8)
```

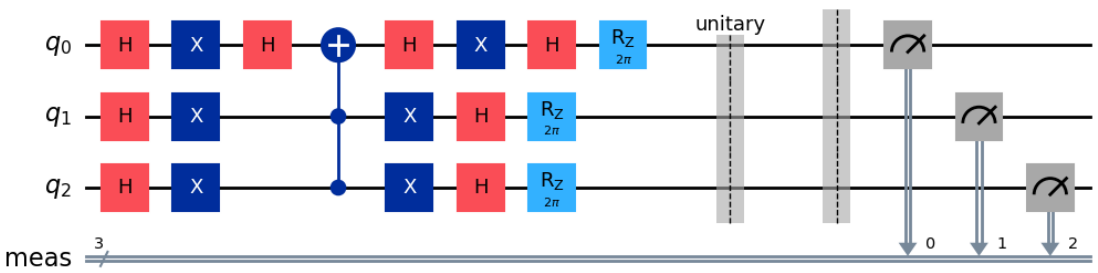
Out[151…

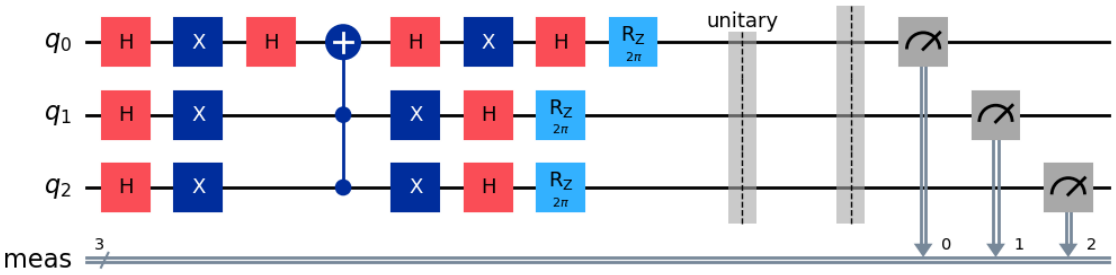
$$\text{G3 =}\begin{bmatrix} -\frac{3}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \\ \frac{1}{4} & -\frac{3}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \\ \frac{1}{4} & \frac{1}{4} & -\frac{3}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \\ \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & -\frac{3}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \\ \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & -\frac{3}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \\ \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & -\frac{3}{4} & \frac{1}{4} & \frac{1}{4} \\ \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & -\frac{3}{4} & \frac{1}{4} \\ \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & -\frac{3}{4} \end{bmatrix}$$

In [152…

```
qc.measure_all()
qc.draw()
```

Out[152…



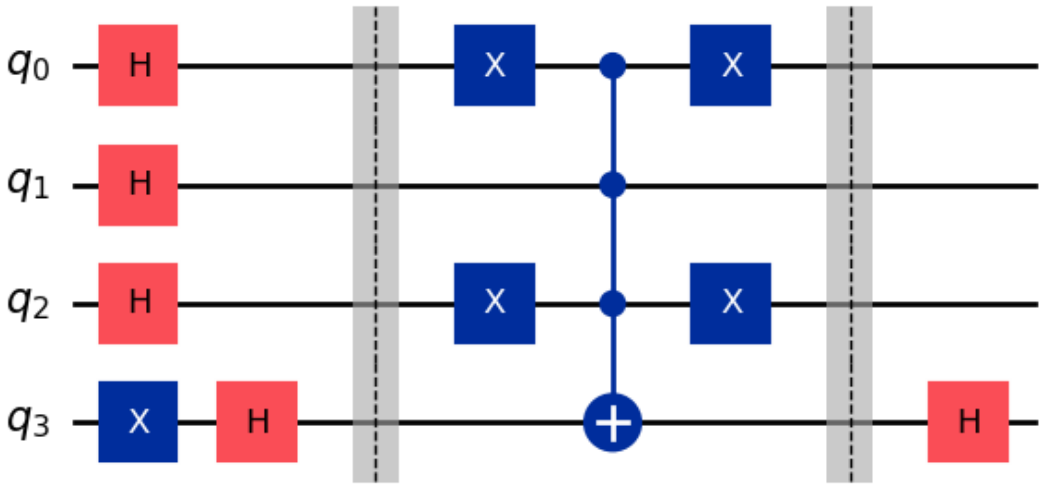


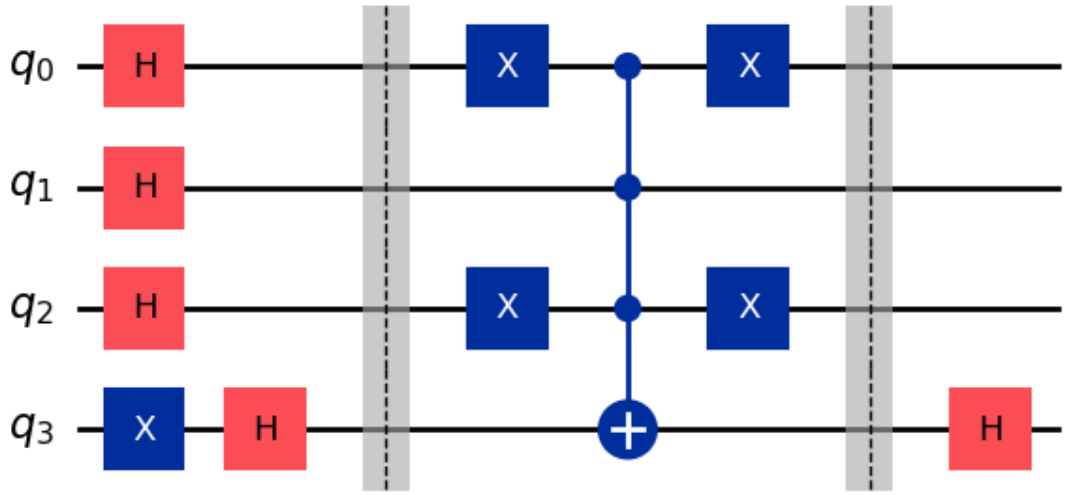
Grover's amplification

In [153…

```
qc = QuantumCircuit(4)
qc.h([0,1,2])
qc.x(3)
qc.h(3)
qc.barrier()
qc.x(0)
qc.x(2)
gate = MCXGate(3)
qc.append(gate, [0, 1, 2, 3])
qc.x(0)
qc.x(2)
qc.barrier()
qc.h(3)
qc.draw()
```

Out[153…





In [154…

```
simulator = Aer.get_backend('statevector_simulator')
final_state = simulator.run(qc).result().get_statevector()
```

In [155…

```
array_to_latex(final_state,max_size=16,prefix="\\text{Statevector} = ")
```

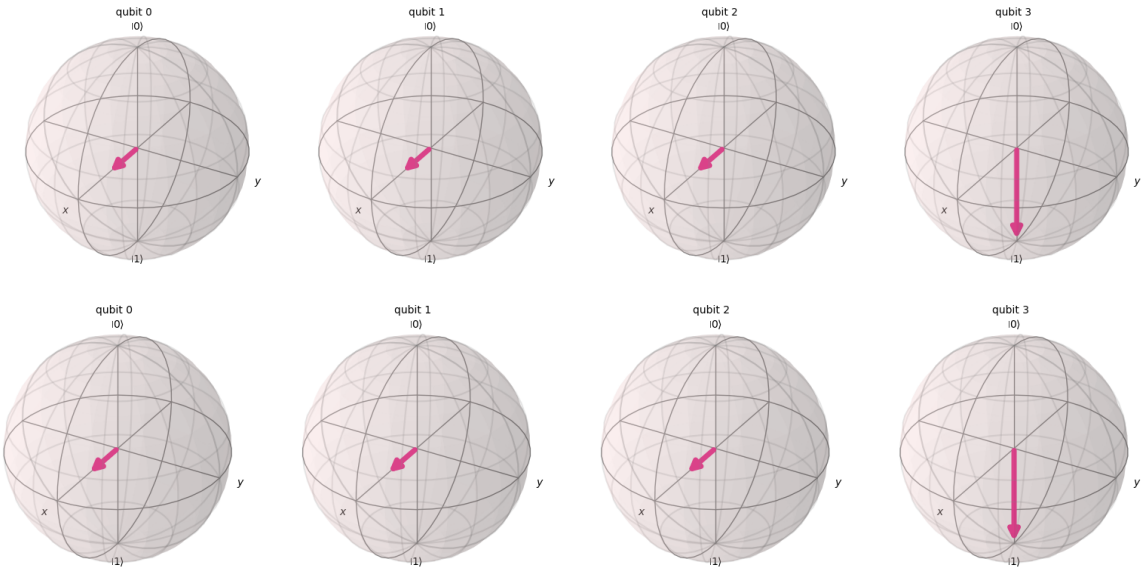
Out[155…

$$\text{Statevector} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \frac{\sqrt{2}}{4} & \frac{\sqrt{2}}{4} & -\frac{\sqrt{2}}{4} & \frac{\sqrt{2}}{4} & \frac{\sqrt{2}}{4} & \frac{\sqrt{2}}{4} \end{bmatrix}$$

In [156…

```
plot_bloch_multivector(final_state)
```

Out[156…

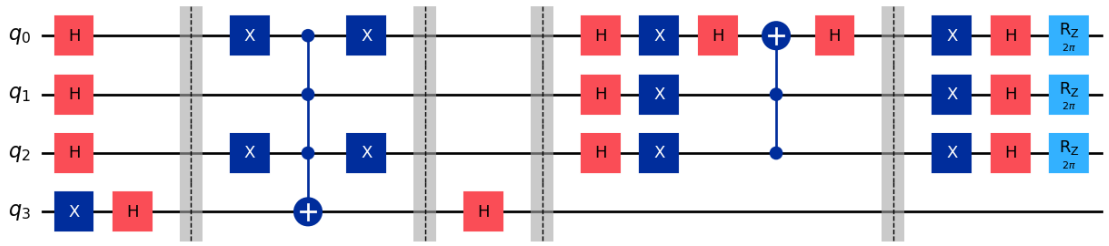


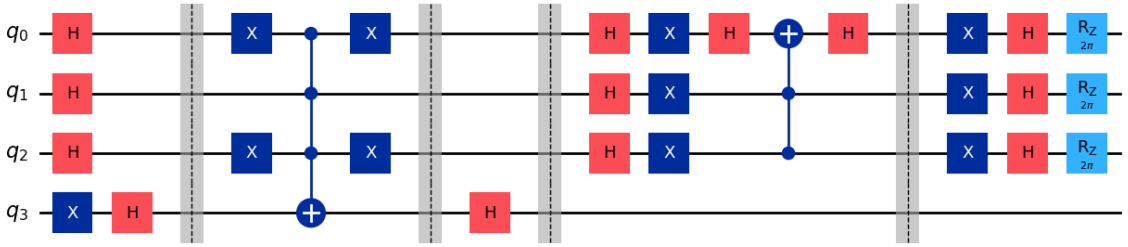
In [157…

```
qc = QuantumCircuit(4)
qc.h([0,1,2])
#Preparation of Aux
qc.x(3)
qc.h(3)
#Oracle
qc.barrier()
qc.x(0)
qc.x(2)
gate = MCXGate(3)
qc.append(gate, [0, 1, 2, 3])
```

```
qc.x(0)
qc.x(2)

#Diffusor
qc.barrier()
qc.h(3)
qc.barrier()
qc.h([0,1,2])
qc.x([0,1,2])
qc.h(0)
qc.ccx(1,2,0)
qc.h(0)
qc.barrier()
qc.x([0,1,2])
qc.h([0,1,2])
#Corrrect the sign, not required...
qc.rz(2*pi,0)
qc.rz(2*pi,1)
qc.rz(2*pi,2)
qc.draw(fold=140)
```

Out[157…





In [158…

```
simulator = Aer.get_backend('statevector_simulator')
final_state = simulator.run(qc).result().get_statevector()
array_to_latex(final_state,max_size=16,prefix="\\text{Statevector} = ")
```

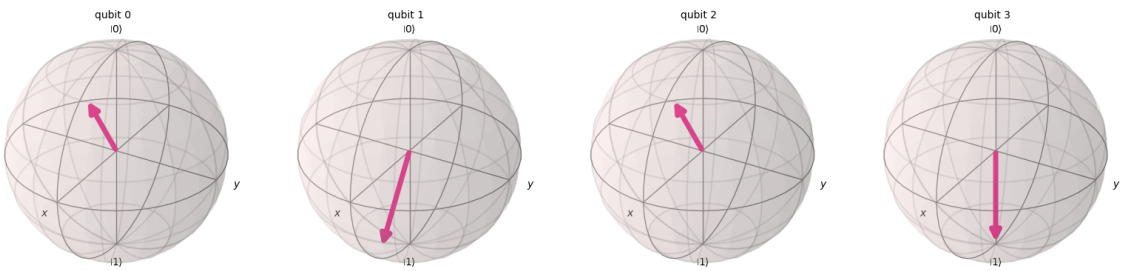
Out[158…

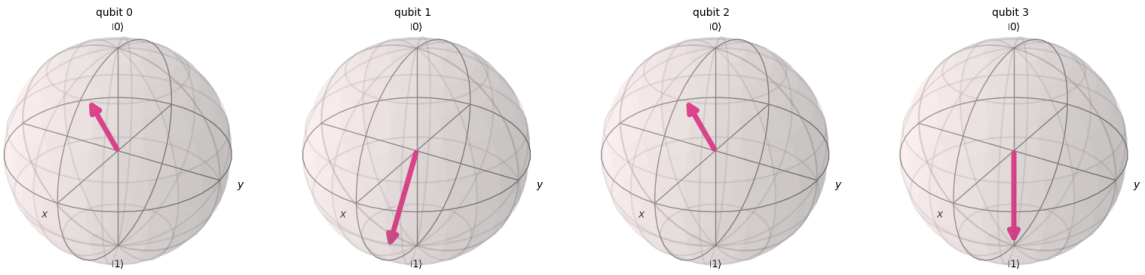
$$\text{Statevector} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0.1767766953 & 0.1767766953 & 0.88 \end{bmatrix}$$

In [159…

```
plot_bloch_multivector(final_state)
```

Out[159…



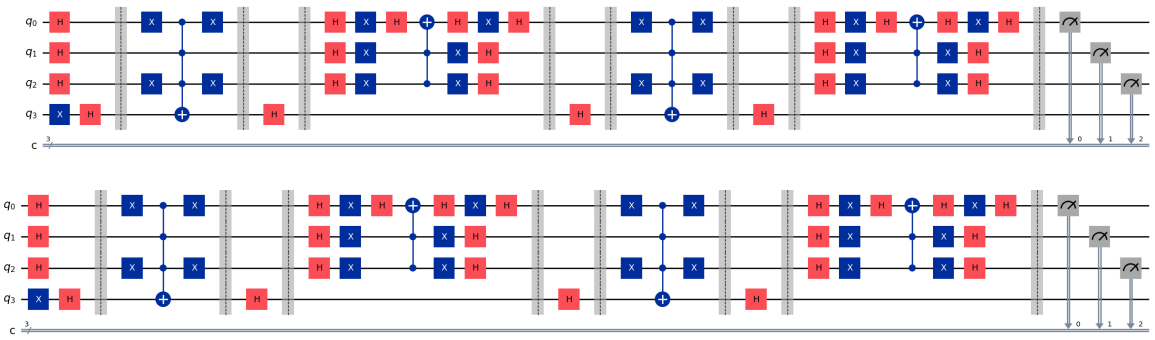


In [160…

```
qc = QuantumCircuit(4,3)
qc.h([0,1,2])
#Preparation of Aux
qc.x(3)
qc.h(3)
#Oracle
qc.barrier()
qc.x(0)
qc.x(2)
gate = MCXGate(3)
qc.append(gate, [0, 1, 2, 3])
qc.x(0)
qc.x(2)
#Diffusor
qc.barrier()
qc.h(3)
qc.barrier()
qc.h([0,1,2])
qc.x([0,1,2])
qc.h(0)
qc.ccx(1,2,0)
qc.h(0)
qc.x([0,1,2])
qc.h([0,1,2])
qc.barrier()
qc.h(3)
#Oracle
qc.barrier()
qc.x(0)
qc.x(2)
gate = MCXGate(3)
qc.append(gate, [0, 1, 2, 3])
qc.x(0)
qc.x(2)
qc.barrier()
qc.h(3)
#Diffusor
qc.barrier()
qc.h([0,1,2])
qc.x([0,1,2])
qc.h(0)
qc.ccx(1,2,0)
qc.h(0)
qc.x([0,1,2])
qc.h([0,1,2])
qc.barrier()
qc.measure(0,0)
qc.measure(1,1)

qc.measure(2,2)
```

```
qc.draw(fold=100)
```

Out[160…



In [161…

```
simulator = Aer.get_backend('statevector_simulator')
final_state = simulator.run(qc).result().get_statevector()
array_to_latex(final_state,max_size=16,prefix="\\text{Statevector} = ")
```

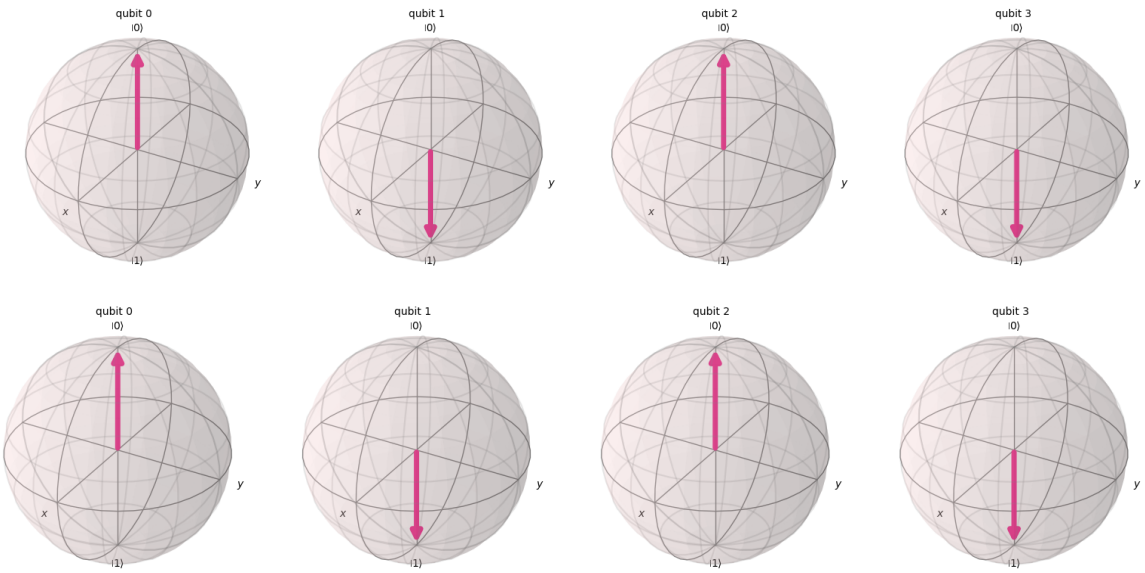
Out[161…

$$\text{Statevector} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

In [162…

```
plot_bloch_multivector(final_state)
```

Out[162…



In [ ]:

In [2]:

```
qc = QuantumCircuit(16)
#Initial state b,a,c,d,c
#First Position 01 is b 01
qc.x(0)
#Second Position 23 is a 00
#Third Position 45 is c 10
qc.x(5)
#Fourth Position 67 is d 11
qc.x(6)
qc.x(7)
#Fifth Position 89 is c 10
qc.x(9)
#Path Descriptor
qc.h(11)
qc.h(12)
qc.barrier()
```

```
#First Rule
#Set flag 10 dependent on the path descriptor
qc.ccx(11,12,10)
# Move
qc.cswap(10,0,2)
qc.cswap(10,1,3)
#Reset flag
qc.ccx(11,12,10)
#Second Rule
#Set flag 10 dependent on the path descriptor
qc.x(11)
qc.ccx(11,12,10)
# Move
qc.cswap(10,2,4)
qc.cswap(10,3,5)
#Reset flag
qc.ccx(11,12,10)
qc.x(11)
#Third Rule
#Set flag 10 dependent on the path descriptor
qc.x(12)
qc.ccx(11,12,10)
# Move
qc.cswap(10,4,6)
qc.cswap(10,5,7)
#Reset flag
qc.ccx(11,12,10)
qc.x(12)
#Fourth Rule
#Set flag 10 dependent on the path descriptor
qc.x(11)
qc.x(12)
qc.ccx(11,12,10)
# Move
qc.cswap(10,6,8)
qc.cswap(10,7,9)
#Reset flag
qc.ccx(11,12,10)
qc.x(12)
qc.x(11)
qc.barrier()
#Depth two: 2th Path Descriptor
qc.h(13)
qc.h(14)
qc.barrier()
#First Rule
#Set flag 10 dependent on the path descriptor
qc.ccx(13,14,10)
# Move
qc.cswap(10,0,2)
qc.cswap(10,1,3)
#Reset flag
qc.ccx(13,14,10)
#Second Rule
#Set flag 10 dependent on the path descriptor
qc.x(13)
qc.ccx(13,14,10)
# Move
qc.cswap(10,2,4)
```

```
qc.cswap(10,3,5)
#Reset flag
qc.ccx(13,14,10)
qc.x(13)
#Third Rule
#Set flag 10 dependent on the path descriptor
qc.x(14)
qc.ccx(13,14,10)
# Move
qc.cswap(10,4,6)
qc.cswap(10,5,7)
#Reset flag
qc.ccx(13,14,10)
qc.x(14)
#Fourth Rule
#Set flag 10 dependent on the path descriptor
qc.x(13)
qc.x(14)
qc.ccx(13,14,10)
# Move
qc.cswap(10,6,8)
qc.cswap(10,7,9)
#Reset flag
qc.ccx(13,14,10)
qc.x(14)
qc.x(13)
qc.barrier()
#Oracle
gate = MCXGate(5)
#Mark the goal state
#Initial state a,b,c,c,d
qc.append(gate, [2,5,7,8,9,15])
```

Out[2]: <qiskit.circuit.instructionset.InstructionSet at 0x79fe355a95d0>

In [3]:

```
def rules():
    qc = QuantumCircuit(15)
    #First Rule
    #Set flag 10 dependent on the path descriptor
    qc.ccx(11,12,10)
    # Move
    qc.cswap(10,0,2)
    qc.cswap(10,1,3)
    #Reset flag
    qc.ccx(11,12,10)
    #Second Rule
    #Set flag 10 dependent on the path descriptor
    qc.x(11)
    qc.ccx(11,12,10)
    # Move
    qc.cswap(10,2,4)
    qc.cswap(10,3,5)
    #Reset flag
    qc.ccx(11,12,10)
    qc.x(11)
    #Third Rule
    #Set flag 10 dependent on the path descriptor
    qc.x(12)
    qc.ccx(11,12,10)
    # Move
```

```
    qc.cswap(10,4,6)
    qc.cswap(10,5,7)
    #Reset flag
    qc.ccx(11,12,10)
    qc.x(12)
    #Fourth Rule
    #Set flag 10 dependent on the path descriptor
    qc.x(11)
    qc.x(12)
    qc.ccx(11,12,10)
    qc.cswap(10,6,8)
    qc.cswap(10,7,9)
    #Reset flag
    qc.ccx(11,12,10)
    qc.x(12)
    qc.x(11)
    #depth two
    #First Rule
    #Set flag 10 dependent on the path descriptor
    qc.ccx(13,14,10)
    # Move
    qc.cswap(10,0,2)
    qc.cswap(10,1,3)
    #Reset flag
    qc.ccx(13,14,10)
    #Second Rule
    #Set flag 10 dependent on the path descriptor
    qc.x(13)
    qc.ccx(13,14,10)
    # Move
    qc.cswap(10,2,4)
    qc.cswap(10,3,5)
    #Reset flag
    qc.ccx(13,14,10)
    qc.x(13)
    #Third Rule
    #Set flag 10 dependent on the path descriptor
    qc.x(14)
    qc.ccx(13,14,10)
    # Move
    qc.cswap(10,4,6)
    qc.cswap(10,5,7)
    #Reset flag
    qc.ccx(13,14,10)
    qc.x(14)
    #Fourth Rule
    #Set flag 10 dependent on the path descriptor
    qc.x(13)
    qc.x(14)
    qc.ccx(13,14,10)
    # Move
    qc.cswap(10,6,8)
    qc.cswap(10,7,9)
    #Reset flag
    qc.ccx(13,14,10)
    qc.x(14)
    qc.x(13)
    qc.name="RULES"

    return qc
```

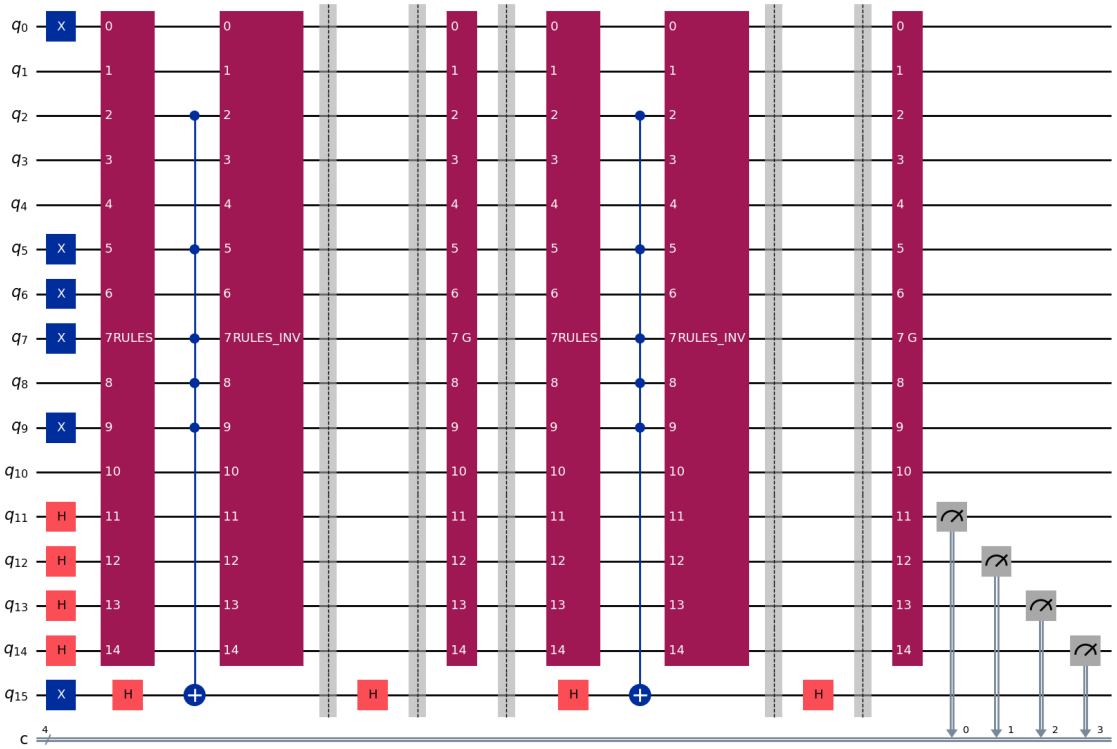
In [4]:
```
def rules_inv():
    qc=rules()
    qc_inv=qc.inverse()
    qc_inv.name="RULES_INV"

    return qc_inv
```

In [5]:
```
def Grover():
    qc = QuantumCircuit(15)
    #Diffusor 11, 12, 13, 14
    qc.h([11,12,13,14])
    qc.x([11,12,13,14])
    qc.h(11)
    gate = MCXGate(3)
    qc.append(gate, [12,13,14,11])
    qc.h(11)
    qc.x([11,12,13,14])
    qc.h([11,12,13,14])
    qc.name="G"

    return qc
```

In [6]:
```
qc = QuantumCircuit(16,4)
qc.x(0)
qc.x(5)
qc.x(6)
qc.x(7)
qc.x(9)
#Path Descriptor
qc.h(11)
qc.h(12)
qc.h(13)
qc.h(14)
#Preparation of Aux
qc.x(15)
qc.h(15)
qc.append(rules(),range(15))
#Oracle
gate = MCXGate(5)
#Mark the goal state
#Initial state a,b,c,c,d
qc.append(gate, [2,5,7,8,9,15])
qc.append(rules_inv(),range(15))
qc.barrier()
qc.h(15)
qc.barrier()
qc.append(Grover(),range(15))
qc.barrier()
qc.h(15)
qc.append(rules(),range(15))
#Oracle
gate = MCXGate(5)
#Mark the goal state
#Initial state a,b,c,c,d
qc.append(gate, [2,5,7,8,9,15])
qc.append(rules_inv(),range(15))
qc.barrier()
qc.h(15)
```

```
qc.barrier()
qc.append(Grover(),range(15))
qc.measure(11,0)
qc.measure(12,1)
qc.measure(13,2)
qc.measure(14,3)
qc.draw(fold=500)
```

Out[6]:

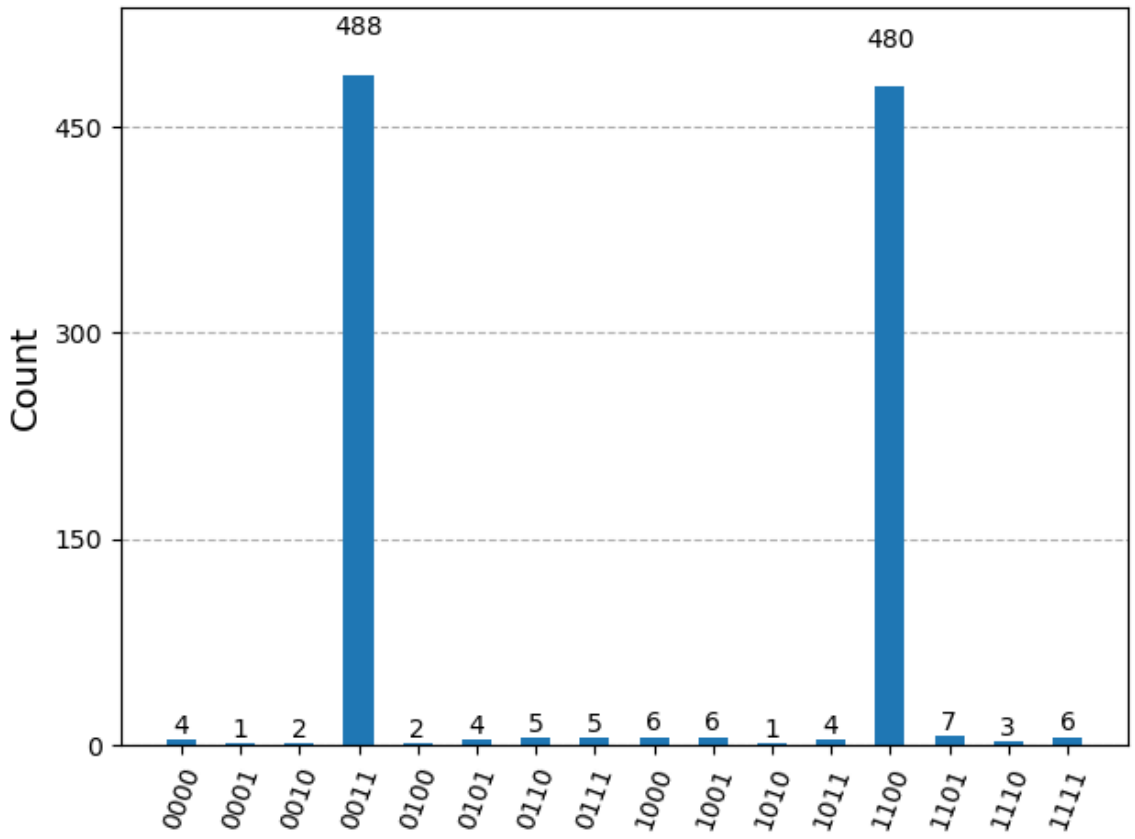


In [7]:

```
simulator = Aer.get_backend('qasm_simulator')
qobj = transpile(qc, simulator)
counts = simulator.run(qobj).result().get_counts()
print("\nTotal count are:",counts)
plot_histogram(counts)
```

```
Total count are: {'0001': 1, '0010': 2, '0000': 4, '1011': 4, '0011': 488,
'0111': 5, '1100': 480, '1101': 7, '1000': 6, '0110': 5, '1111': 6, '1001'
: 6, '0101': 4, '1110': 3, '1010': 1, '0100': 2}
```

Out[7]:



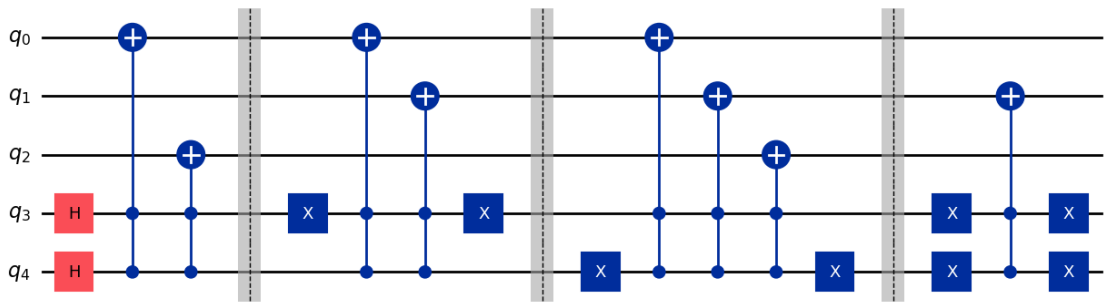
ENTANGLEMENT OF BINARY PATTERNS

In [19]:

```
qc = QuantumCircuit(5)
#0-2 data
#Index
#3-4
qc.h(3)
qc.h(4)
#First patern
qc.ccx(3,4,0)
qc.ccx(3,4,2)
qc.barrier()
#Second patern
qc.x(3)
qc.ccx(3,4,0)
qc.ccx(3,4,1)
qc.x(3)
qc.barrier()
#Third patern
qc.x(4)
qc.ccx(3,4,0)
qc.ccx(3,4,1)
qc.ccx(3,4,2)
qc.x(4)
qc.barrier()
#Fourth patern
qc.x(3)
qc.x(4)
qc.ccx(3,4,1)
qc.x(4)
qc.x(3)
```

```
.draw()
```

Out[19]:

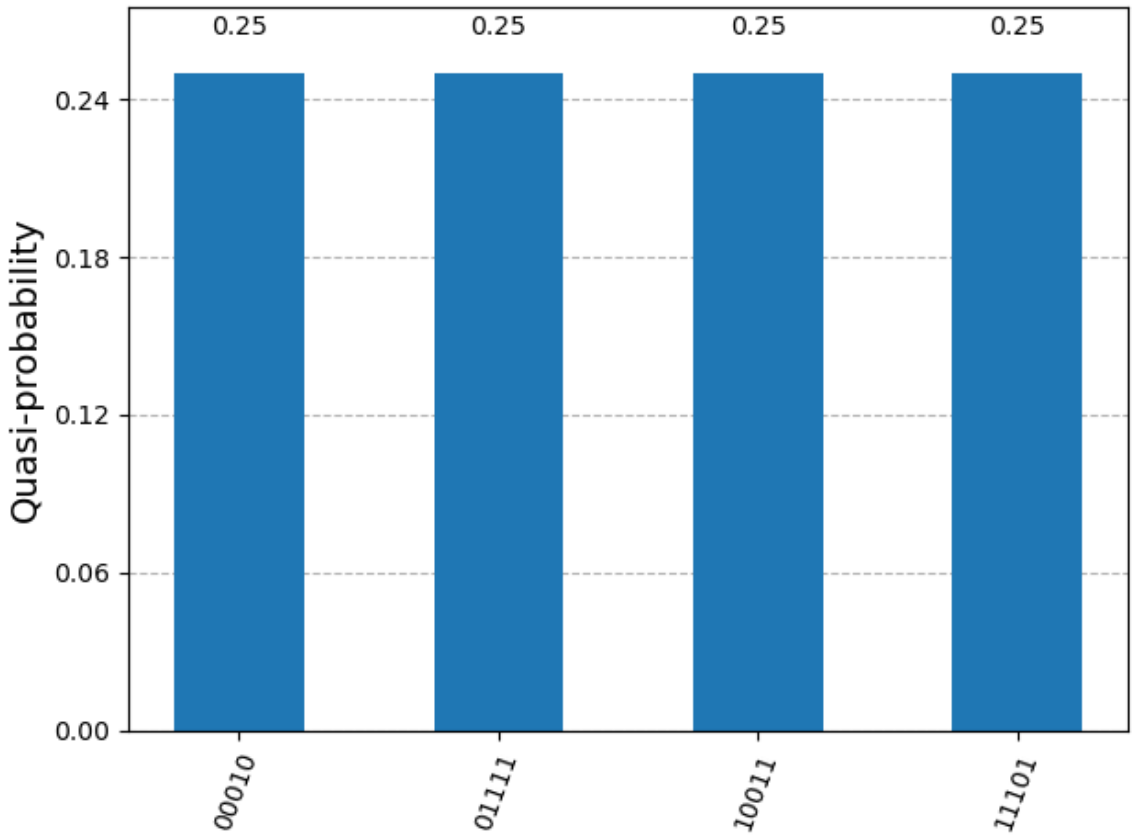


In [20]:

```
simulator = Aer.get_backend('statevector_simulator')
job = simulator.run(qc, shots=1000)
result = job.result()
counts = result.get_counts()
print("\nCounts:", counts)
plot_histogram(counts)
```

```
Counts: {np.str_('00010'): np.float64(0.25), np.str_('01111'): np.float64
(0.25), np.str_('10011'): np.float64(0.25), np.str_('11101'): np.float64
(0.25)}
```

Out[20]:



In [21]:

```
simulator = Aer.get_backend('statevector_simulator')
final_state = simulator.run(qc).result().get_statevector()
array_to_latex(final_state,max_size=16,prefix="\\text{Statevector} = ")
```

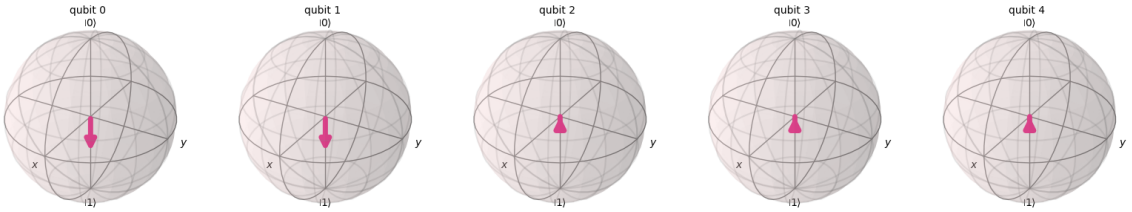
Out[21]:

$$\text{Statevector} = \begin{bmatrix} 0 & 0 & \frac{1}{2} & 0 & 0 & 0 & 0 & 0 & \cdots & 0 & 0 & 0 & 0 & \frac{1}{2} & 0 & 0 \end{bmatrix}$$

In [22]:

```
plot_bloch_multivector(final_state)
```

Out[22]:


qubit 0
|0⟩
|1⟩
x
y
qubit 1
|0⟩
|1⟩
x
y
qubit 2
|0⟩
|1⟩
x
y
qubit 3
|0⟩
|1⟩
x
y
qubit 4
|0⟩
|1⟩
x
y

In [ ]: